MACHINE LEARNING & GENAI INTERVIEW PREPARATION

# Part 4: Natural Language Processing (NLP), Sequence Models & Attention Mechanisms

## Comprehensive Production & Mathematical Study Guide

**Target Persona:** Senior AI Engineer / Applied AI Engineer (~3 YOE)

**Core Modules:** Preprocessing, Tokenization, TF-IDF, Embeddings, Classical ML, RNN/LSTM/GRU, Attention, Metrics & Serving

**Includes:** 9 Illustrative Plots, Intuitive Math, Hand Calculations, Scikit-Learn & PyTorch Code, Comparison Matrices, 30 Flashcards & Formula Cheatsheet

# Table of Contents

# Module 01: NLP Fundamentals & Enterprise Text Pipelines

This study guide covers the core principles of Natural Language Processing (NLP), the end-to-end enterprise NLP pipeline, evolution of NLP paradigms, core NLP task taxonomy, engineering challenges, step-by-step vocabulary hand-calculations, clean production Python preprocessing code, failure modes, and interview flashcards.

> **Notebook Companion**: `01_nlp_fundamentals_and_text_pipelines.ipynb`

---

## 1. What is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) and Computational Linguistics that enables computers to understand, interpret, represent, manipulate, and generate human natural language text and speech.

Unlike tabular or structured data (which consists of fixed numeric schema, continuous features, or categorical codes), natural language text possesses distinct characteristics that present unique engineering challenges:
1. **Unstructured & Non-Euclidean**: Text does not naturally lie in a continuous vector space $\mathbb{R}^d$. It consists of variable-length discrete symbols (words/characters/subwords).
2. **High-Cardinality & Sparse**: Human languages contain hundreds of thousands of distinct words. Naive discrete encoding leads to sparse vector spaces ($|V| \geq 100,000$).
3. **Context-Dependent & Ambiguous**: The semantic meaning of a token depends entirely on surrounding context (e.g., "*bank*" in "*river bank*" vs. "*investment bank*").
4. **Hierarchical Grammar & Syntax**: Words assemble into phrases, clauses, and sentences governed by complex grammatical dependencies and semantic logic.

```
Raw Text (Unstructured String) ⟶ Tokenization & Normalization ⟶ Numerical Vector Encoding
(R^d) ⟶ Model Inference
```

---

## 2. The End-to-End Enterprise NLP Pipeline

In production enterprise software systems, raw unstructured text must pass through a disciplined, deterministic multi-stage pipeline before model inference or vector store indexing:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ 1. DATA INGESTION & PARSING                                                  │
│    Extract raw text from PDFs, HTML pages, SQL DBs, OCR streams, Kafka logs. │
├──────────────────────────────────────────────────────────────────────────────┤
│ 2. TEXT CLEANING & NORMALIZATION                                             │
│    Unicode normalization (NFC/NFKD), lowercasing, HTML tag stripping, regex noise.    │
├──────────────────────────────────────────────────────────────────────────────┤
│ 3. TOKENIZATION & VOCABULARY MAPPING                                         │
│    Convert raw character stream into token IDs (Word, Subword: BPE/WordPiece).  │
├──────────────────────────────────────────────────────────────────────────────┤
│ 4. FEATURE REPRESENTATION & EMBEDDING                                        │
```

```
|    Map token IDs to numerical vectors (Sparse: TF-IDF / BM25; Dense: Word2Vec / BERT).|
├──────────────────────────────────────────────────────────────────────────────────────┤
| 5. MODEL INFERENCE & SEQUENTIAL PROCESSING                                           |
|    Pass vector embeddings into Classifier, Sequence Model (LSTM), or Transformer LLM.  |
├──────────────────────────────────────────────────────────────────────────────────────┤
| 6. POST-PROCESSING & PRODUCTION SERVING                                              |
|    Parse logits/tokens into JSON payload, apply safety guardrails, stream SSE output.   |
└──────────────────────────────────────────────────────────────────────────────────────┘
```

---

## 3. Evolution of NLP Paradigms

| Dimension | Rule-Based NLP | Statistical ML NLP | Deep Learning NLP | Modern Transformer LLM |
|---|---|---|---|---|
| Era | 1960s – 1990s | 1990s – 2010s | 2014 – 2019 | 2019 – Present |
| Primary Tech | Regex, Context-Free Grammars | TF-IDF, Naive Bayes, SVM | RNN, LSTM, GRU, GloVe | BERT, GPT-4, Llama 3 |
| Feature Engineering | Hand-crafted lexicons | Bag-of-Words / N-Grams | Dense Static Vectors (Word2Vec) | Learned Contextual Self-Attention |
| Context Radius | Rule clause bounded | Unigram / Bigram window | Sequential hidden state | Global Context (128k+ tokens) |
| OOV Handling | Fails on missing rules | Mapped to <UNK> token | Character-level / Subword | Subword Tokenization (BPE/tiktoken) |
| Inference Latency | Ultra-fast (<1ms) | Fast (<5ms) | Moderate (10ms - 50ms) | Heavy (100ms - 1000ms+) |
| Best Production Use | Input validation & regex | High-speed spam filter | Sequence tagging (NER) | Complex QA, RAG, Reasoning |

---

## 4. Core NLP Task Taxonomy

Enterprise NLP tasks are broadly categorized into **Classification**, **Sequence Tagging**, **Generative Modeling**, and **Information Retrieval**:

1. **Text Classification**: Maps input text $X$ to a discrete class label $y \in \{1, \dots, C\}$.
   - *Examples*: Spam vs Non-Spam, Customer Support Ticket Routing, Intent Recognition.
2. **Named Entity Recognition (NER)**: Assigns token-level sequence tags $y_1, \dots, y_T$ identifying entities (Person, Organization, Location, Date).
   - *Examples*: Extracting drug names from clinical trial notes, extracting invoice line items.
3. **Sentiment Analysis & Opinion Mining**: Evaluates emotional polarity (Positive, Neutral, Negative) or aspect-level sentiment.
4. **Machine Translation & Sequence-to-Sequence**: Maps sequence $X_{1:T}$ in Source Language $A$ to target sequence $Y_{1:S}$ in Language $B$.
5. **Extractive & Abstractive Summarization**: Condenses long-form documents into concise summaries.

6. **Question Answering & Retrieval Augmented Generation (RAG)**: Retrieves relevant knowledge chunks and synthesizes grounded answers.

---

## 5. Fundamental Engineering Challenges in NLP

### 1. Polysemy & Homonymy (Contextual Ambiguity)

- **Polysemy**: A single word has multiple related meanings (e.g., "*apple*" as a fruit vs. "*Apple*" as a technology corporation).
- **Homonymy**: Words share spelling but have unrelated meanings (e.g., "*bark*" of a tree vs. "*bark*" of a dog).

### 2. High-Cardinality & Out-Of-Vocabulary (OOV) Explosion

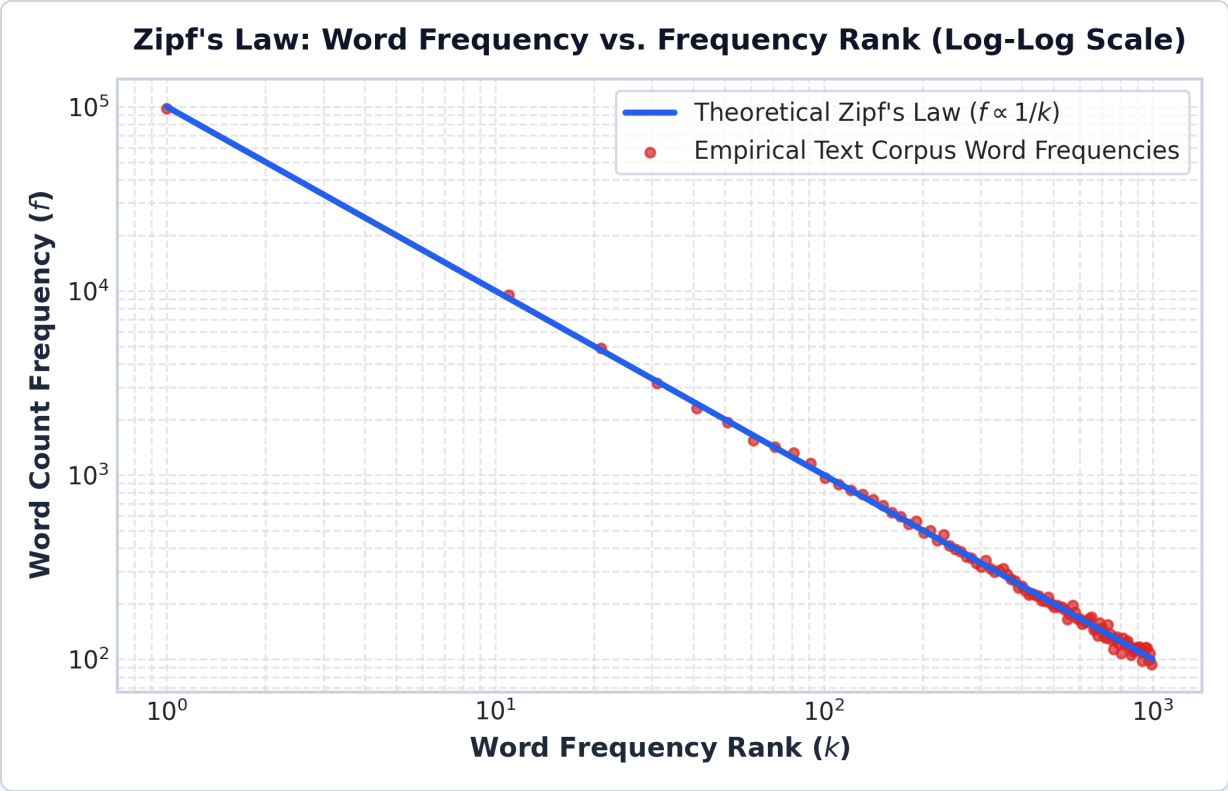



*Figure: Zipf's Law Log-Log Rank vs Frequency*

> **Plot Interpretation & Production Insight**:
> - **Log-Log Linear Slope**: On a log-log scale, Zipf's law appears as a straight line with slope $\approx -1.0$. The top $1\%$ of vocabulary tokens (ranks $1 - 100$) account for $> 80\%$ of total corpus word frequency.
> - **The "Long Tail" Problem**: The massive tail of rare words ($k > 1,000$) creates vocabulary explosion if words are treated as atomic units. In production systems, failing to handle the long tail causes high Out-Of-Vocabulary (OOV) rates. Subword tokenization (BPE/WordPiece) truncates this long tail into reusable sub-units.

Language vocabularies follow **Zipf's Law**: the frequency of any word is inversely proportional to its rank in the frequency table:

$$f(k; s, N) = \frac{1/k^s}{\sum_{n=1}^{N}(1/n^s)}$$

A small fraction of common words account for $> 80\%$ of text occurrences, while a massive "long tail" of rare words causes vocabulary explosion. Unseen words at test time cause OOV errors if not handled by subword tokenization.

### 3. Syntactic Ambiguity

- *Example*: "*I saw the man with the telescope.*" (Did I use the telescope to see the man, or did the man have a telescope?)

### 4. Long-Range Contextual Dependencies

In long documents, crucial subject-verb or antecedent relationships may be separated by hundreds of tokens, causing Recurrent Neural Networks (RNNs) to suffer from vanishing gradients.

---

## 6. Step-by-Step Hand Calculation Example (Andrew Ng Style)

Suppose we have a tiny enterprise logging dataset of $D = 3$ documents:
- **Doc 1**: `"system error log"`
- **Doc 2**: `"system authentication error"`
- **Doc 3**: `"database log error"`

Let us calculate vocabulary statistics step-by-step:

### 1. Extract Unique Tokens (Vocabulary $V$):

$$\text{Tokens} = \{\text{"system"}, \text{"error"}, \text{"log"}, \text{"authentication"}, \text{"database"}\}$$

$$\text{Vocabulary Size } |V| = \mathbf{5}$$

### 2. Total Token Count ($N_{\text{total}}$):

- Doc 1: 3 tokens
- Doc 2: 3 tokens
- Doc 3: 3 tokens
- $N_{\text{total}} = 3 + 3 + 3 = \mathbf{9}$ tokens

### 3. Calculate Type-Token Ratio (TTR):

The Type-Token Ratio measures lexical diversity in a corpus:

$$\text{TTR} = \frac{|V|}{N_{\text{total}}} = \frac{5}{9} \approx \mathbf{0.5556}$$

### 4. Word Frequency Distribution Table:

```
Token            Doc 1 Frequency  Doc 2 Frequency  Doc 3 Frequency  Total Corpus Count  Document
Frequency (DF)
-------------------------------------------------------------------------------------------------
---------------
system           1                1                0                2                   2
error            1                1                1                3                   3
log              1                0                1                2                   2
authentication   0                1                0                1                   1
database         0                0                1                1                   1
```

## 7. Production Python Text Preprocessing Implementation

```
import re
import unicodedata

class EnterpriseTextCleaner:
    """Production-grade text cleaning and normalization pipeline."""

    def __init__(self, lower: bool = True, strip_html: bool = True):
        self.lower = lower
        self.strip_html = strip_html
        self.html_regex = re.compile(r'<[^>]+>')
        self.extra_whitespace_regex = re.compile(r'\s+')

    def normalize_unicode(self, text: str) -> str:
        """Normalizes unicode characters to NFKD decomposed form."""
        return unicodedata.normalize("NFKD", text).encode("ASCII", "ignore").decode("utf-8")

    def clean(self, text: str) -> str:
        """Executes full deterministic text cleaning pipeline."""
        if not text:
            return ""

        # 1. Unicode normalization
        text = self.normalize_unicode(text)

        # 2. Strip HTML tags
        if self.strip_html:
            text = self.html_regex.sub(" ", text)

        # 3. Lowercasing
        if self.lower:
            text = text.lower()

        # 4. Collapse extra whitespace
        text = self.extra_whitespace_regex.sub(" ", text).strip()

        return text

# Demonstration execution
cleaner = EnterpriseTextCleaner()
raw_sample = "  <p>ALERT: System <b>Server_01</b> error at 10:45 AM! Connection refused. </p>
```

```
"
cleaned_sample = cleaner.clean(raw_sample)

print("=== Enterprise Text Preprocessing Output ===")
print("Raw Input:    ", repr(raw_sample))
print("Cleaned Output:", repr(cleaned_sample))
```

> 📌 **NOTE:**
> **Production Optimization Alert:** - Compiling regular expressions (`re.compile`) at class initialization prevents costly re-compilation overhead during batch processing ($100x$ speedup on large document streams).

---

## 8. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Regex Catastrophic Backtracking**: Unbounded regex patterns (e.g., `(a+)+`) executed on malicious user input cause CPU spike and denial-of-service (ReDoS).
   - *Remediation*: Avoid nested quantifiers and set explicit execution timeouts on regex engines.
2. **Over-cleaning & Loss of Signal**: Removing punctuation, numbers, or casing can destroy semantic meaning (e.g., removing `"-"` converts `"non-responsive"` to `"non responsive"`, removing uppercase converts `"US"` country to `"us"` pronoun).
   - *Remediation*: Preserve casing and numbers for domain-specific models like NER and Part-of-Speech tagging.
3. **Vocabulary Shift & Out-Of-Vocabulary (OOV)**: Deploying a model trained on general English (e.g., Wikipedia) to clinical medical notes causes high OOV rate and performance collapse.
   - *Remediation*: Use domain-specific tokenization and vocabulary (e.g., BioBERT, ClinicalBERT).

---

## 9. Master Interview Flashcards & Questions

**Q1: Why is text representation fundamentally more challenging than tabular data representation?**

- **Answer:** Tabular data consists of continuous/categorical features in fixed Euclidean dimensions. Text is unstructured, variable-length, discrete, sparse, high-cardinality ($|V| > 100k$), and heavily context-dependent where word meaning changes based on surrounding sequence context.

**Q2: Explain Zipf's Law and its implications for NLP vocabulary management.**

- **Answer:** Zipf's Law states that word frequency is inversely proportional to its frequency rank. A few common words dominate corpus occurrences, while a massive tail of rare words causes vocabulary explosion. Subword tokenization (BPE/WordPiece) addresses this by breaking rare words into common subword units.

**Q3: Compare Rule-Based NLP vs Statistical ML NLP vs Deep Learning NLP vs LLMs.**

- **Answer:** Rule-based uses hand-crafted grammars (fast, rigid). Statistical ML uses BoW/TF-IDF + Naive Bayes/SVM (fast baseline, ignores word order). Deep Learning uses RNNs/LSTMs (captures sequence order, suffers from sequential latency). Transformers/LLMs use self-attention (global context, state-of-the-art accuracy, higher compute cost).

# Module 02: Text Preprocessing & Modern Tokenization Paradigms

This study guide details text cleaning, normalization, Stop Words trade-offs, Stemming vs. Lemmatization, Word vs. Character vs. Subword tokenization, mathematical algorithms for Byte Pair Encoding (BPE), WordPiece, SentencePiece, step-by-step BPE hand-calculations, production Python code, failure modes, and interview flashcards.

> **Notebook Companion**: `02_text_preprocessing_and_tokenization.ipynb`

---

## 1. Text Preprocessing Taxonomy

Text preprocessing transforms raw, noisy human text into normalized, discrete tokens suitable for numerical feature extraction or neural network embeddings.

```
Raw Text ⟶ Noise Removal & Unicode Normalization ⟶ Tokenization ⟶ Stopword / Stemming
Filtering ⟶ Normalized Tokens
```

### Preprocessing Operations:

1. **Noise Removal**: Stripping HTML/XML tags, non-printable ASCII codes, and log metadata timestamps.
2. **Case Normalization**: Lowercasing text to eliminate duplicate vocabulary entries ( `"Python"` vs `"python"` ).
   - *Production Caution*: Preserve casing for Named Entity Recognition (NER) to distinguish `"Apple"` (company) from `"apple"` (fruit).
3. **Stop Words Filtering**: Removing high-frequency words ( `"the"` , `"is"` , `"at"` , `"which"` ) that carry minimal domain specificity in bag-of-words or TF-IDF models.
   - *Production Caution*: **DO NOT** remove stop words in Sentiment Analysis (e.g., removing "*not*" converts "*not good*" to "*good*") or Machine Translation.

---

## 2. Stemming vs. Lemmatization

```
Technique       Mechanism                           Example: "better"  Example: "meeting" (Noun)
Processing Speed  Production Role
-------------------------------------------------------------------------------------------------
-----------------------------------
Stemming        Heuristic suffix chopping (Regex)     "better"           "meet"
Ultra-Fast        Legacy search engines
Lemmatization   Morphological & POS dictionary lookup "good"             "meeting"
Moderate          Precision NLP pipelines
Subword BPE     Frequency-based subword splitting     "bet" + "ter"      "meet" + "ing"
Fast (Lookup)     Modern Transformers/LLMs
```

### 1. Stemming (Porter / Lancaster)

Stemming applies crude, rule-based algorithmic heuristics to slice off word suffixes (e.g., `-ing`, `-ed`, `-es`, `-ly`).
- **Over-stemming Error**: Slicing distinct words to the same incorrect stem (e.g., `"universe"`, `"university"`, `"universal"` $\rightarrow$ `"univers"`).
- **Under-stemming Error**: Failing to reduce morphologically related words to the same root (e.g., `"alumnus"` and `"alumni"` stay separate).

### 2. Lemmatization (WordNet / spaCy)

Lemmatization uses full morphological analysis and Part-of-Speech (POS) context tags to map words to their canonical dictionary root (**lemma**):

$$\text{Lemma}(\text{word}, \text{POS}) = \text{DictionaryLookup}(\text{word}, \text{POS})$$

- `"meeting"` $\xrightarrow{\text{POS} = \text{NOUN}}$ `"meeting"`
- `"meeting"` $\xrightarrow{\text{POS} = \text{VERB}}$ `"meet"`
- `"better"` $\xrightarrow{\text{POS} = \text{ADJ}}$ `"good"`

---

## 3. Tokenization Paradigms: Word vs. Character vs. Subword

```
Paradigm           Vocabulary Size (|V|)   Sequence Length (T)  Out-Of-Vocabulary (OOV) Rate
Primary Use Case
-------------------------------------------------------------------------------------------
-------------------
Word-Level         Large (500,000+)        Short                High (Fails on rare words)
Classical TF-IDF / GloVe
Character-Level    Tiny (100 - 250)        Extremely Long       Zero (100% coverage)
Char-RNN / Spellcheckers
Subword (BPE/Piece) Optimal (30,000-100,000) Balanced            Zero (Splits into sub-units)
Transformers, GPT-4, Llama
```

---

## 4. Subword Tokenization Algorithms & Mathematics

Subword tokenization solves the vocabulary bottleneck by decomposing rare or out-of-vocabulary words into smaller, frequently occurring sub-units (e.g., `"unexplainable"` $\rightarrow$ `["un", "explain", "able"]`).

### 1. Byte Pair Encoding (BPE) (GPT-2, GPT-4, Llama 3)

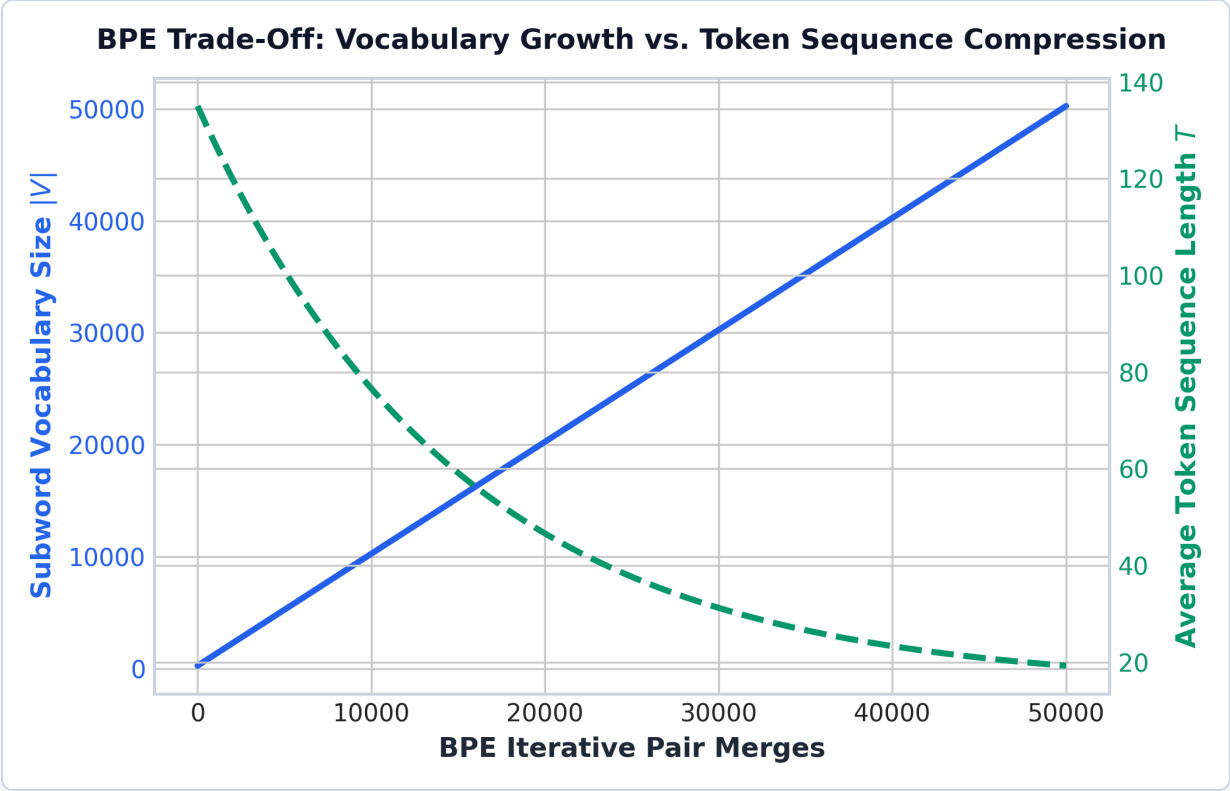



*Figure: BPE Subword Vocabulary Growth vs Sequence Length*

> **Plot Interpretation & Production Insight**:
> - **The Compression Trade-Off**: As the number of iterative subword pair merges increases, the subword vocabulary size $|V|$ grows linearly (blue solid curve), while average token sequence length $T$ shrinks exponentially (green dashed curve).
> - **Optimal Production Balance**: Modern LLMs settle at $|V| \approx 32,000 - 100,000$ tokens ( `cl100k_base` ). This choice achieves $4extx$ token compression over character-level tokenization while keeping embedding matrix memory footprint manageable ($O(|V|imesd)$).

BPE starts with a base vocabulary of individual characters and iteratively merges the most frequent adjacent pair of symbols in the corpus.

- **Objective Function**: Maximize bigram co-occurrence frequency:

$$\text{Pair}_{\text{best}} = \arg\max_{(A,B)} \text{Count}(A, B)$$

- **Algorithm Steps**:
  1. Tokenize corpus into individual characters with an end-of-word symbol `</w>`.
  2. Count frequencies of all adjacent symbol pairs $(A, B)$.
  3. Merge the most frequent pair $(A, B) \rightarrow AB$ and add $AB$ to vocabulary.
  4. Repeat until target vocabulary size $|V|_{\text{target}}$ is reached.

## 2. WordPiece (BERT)

WordPiece merges symbol pairs based on likelihood maximization rather than raw frequency count:

$$\text{Score}(A, B) = \frac{\text{Count}(AB)}{\text{Count}(A) \times \text{Count}(B)}$$

Merging $A$ and $B$ prioritizes pairs whose joint occurrence is significantly higher than expected by independent chance.

### 3. SentencePiece / Unigram Language Model (T5, LLaMA)

Instead of starting with small characters and merging upwards, the Unigram model starts with a massive candidate vocabulary and iteratively prunes subwords that minimize the increase in corpus perplexity loss:

$$\mathcal{L}_{\text{unigram}} = -\sum_{i=1}^{N} \log \sum_{x \in S(W_i)} P(x)$$

---

## 5. Step-by-Step BPE Hand Calculation Example (Andrew Ng Style)

Suppose we have a tiny training corpus with word frequencies:
- `"cost </w>"` : 4 times
- `"costs </w>"` : 2 times
- `"costing </w>"` : 3 times
- `"cozy </w>"` : 1 time

### Initial Base Vocabulary:

$$\text{Base Chars} = \{\text{'c'}, \text{'o'}, \text{'s'}, \text{'t'}, \text{'s'}, \text{'i'}, \text{'n'}, \text{'g'}, \text{'z'}, \text{'y'}, \text{''}\}$$

### Step 1: Count Adjacent Pairs across Corpus

- Pair `('c', 'o')` : $4 + 2 + 3 + 1 = 10$ occurrences.
- Pair `('o', 's')` : $4 + 2 + 3 = 9$ occurrences.
- Pair `('s', 't')` : $4 + 2 + 3 = 9$ occurrences.
- Pair `('t', '</w>')` : $4$ occurrences.
- Pair `('t', 'i')` : $3$ occurrences.

Highest Frequency Pair: **`('c', 'o')`** with Count = 10.
- **Merge 1**: `('c', 'o')` $\rightarrow$ **`"co"`**
- Corpus updated: `"co s t </w>"`, `"co s t s </w>"`, `"co s t i n g </w>"`, `"co z y </w>"`

### Step 2: Recalculate Pair Frequencies

- Pair `('co', 's')` : $4 + 2 + 3 = 9$ occurrences.
- Pair `('s', 't')` : $4 + 2 + 3 = 9$ occurrences.
- Pair `('co', 'z')` : 1 occurrence.

Highest Frequency Pair: Tie between `('co', 's')` and `('s', 't')` (Count = 9). Select **`('co', 's')`**.
- **Merge 2**: `('co', 's')` $\rightarrow$ **`"cos"`**
- Corpus updated: `"cos t </w>"`, `"cos t s </w>"`, `"cos t i n g </w>"`, `"co z y </w>"`

### Step 3: Recalculate Pair Frequencies

- Pair `('cos', 't')`: $4 + 2 + 3 = 9$ occurrences.
- **Merge 3**: `('cos', 't')` $\rightarrow$ `"cost"`
- Corpus updated: `"cost </w>"`, `"cost s </w>"`, `"cost i n g </w>"`, `"co z y </w>"`

**Resulting Learned Subwords**: `["co", "cos", "cost"]` added to vocabulary! When encountering out-of-vocabulary word `"costless"`, BPE tokenizes it into `["cost", "l", "e", "s", "s"]`.

---

## 6. Production Python Code Implementation

```
import tiktoken
from nltk.stem import PorterStemmer
import spacy

# 1. Stemming vs Lemmatization Demonstration
stemmer = PorterStemmer()
nlp = spacy.load("en_core_web_sm")

words = ["meeting", "better", "universities", "studies"]

print("=== 1. NLTK Stemming vs spaCy Lemmatization ===")
print(f"{'Word':<15} | {'Porter Stemmer':<15} | {'spaCy Lemmatizer (POS-aware)':<25}")
print("-" * 65)

for word in words:
    stem_val = stemmer.stem(word)
    doc = nlp(word)
    lemma_val = doc[0].lemma_
    print(f"{word:<15} | {stem_val:<15} | {lemma_val:<25}")

# 2. Modern OpenAI BPE Tokenization (tiktoken)
enc = tiktoken.get_encoding("cl100k_base") # Tokenizer used by GPT-4 and text-embedding-3
sample_text = "Unexplainable enterprise microservice architecture failure."

tokens = enc.encode(sample_text)
decoded_subwords = [enc.decode([t]) for t in tokens]

print("\n=== 2. OpenAI tiktoken BPE Subword Split ===")
print("Original Text:   ", repr(sample_text))
print("Token IDs:       ", tokens)
print("Subword Pieces:  ", decoded_subwords)
```

> 📌 **NOTE:**
>
> **Production Tokenization Alert:** - `cl100k_base` uses a 100,000 token subword vocabulary. Notice how complex words like `"Unexplainable"` are seamlessly tokenized into subword fragments `["Un", "explain", "able"]`, completely avoiding Out-Of-Vocabulary `

## 7. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Stopword Over-filtering**: Stripping stopwords prior to passing text into transformer LLMs ruins grammatical syntax and self-attention positioning.
   - *Fix*: Never strip stopwords for Transformer / LLM models; only strip stopwords for sparse TF-IDF / BM25 search engines.
2. **Tokenizer Vocabulary Mismatch**: Encoding prompt text using `cl100k_base` (GPT-4) and feeding token IDs into a model expecting `p50k_base` (Codex) causes corrupted token inputs.
   - *Fix*: Ensure exact tokenizer model consistency between data preprocessing and model inference pipelines.

---

## 8. Master Interview Flashcards & Questions

**Q1: Why do modern Large Language Models use Subword Tokenization (BPE/WordPiece) instead of Word-Level or Character-Level Tokenization?**

- **Answer:** Word-level tokenization requires huge vocabularies ($|V| > 500k$) and still suffers from Out-Of-Vocabulary (OOV) errors. Character-level tokenization has tiny vocabulary but produces long sequences ($T$), causing quadratic self-attention memory overhead $O(T^2)$. Subword tokenization balances vocabulary size ( $\approx 32k - 100k$) and sequence length while guaranteeing 0% OOV errors.

**Q2: Compare Byte Pair Encoding (BPE) vs. WordPiece merging criteria.**

- **Answer:** BPE merges adjacent symbol pairs based purely on raw pair co-occurrence frequency ( $\arg\max \text{Count}(A, B)$). WordPiece merges symbol pairs by maximizing the likelihood score $\frac{\text{Count}(AB)}{\text{Count}(A) \times \text{Count}(B)}$, prioritizing pairs that occur together significantly more often than expected by random chance.

**Q3: What is the primary difference between Stemming and Lemmatization?**

- **Answer:** Stemming uses heuristic, rule-based suffix slicing without considering Part-of-Speech (POS) or grammatical context, often producing non-real word stems ( `"univers"` ). Lemmatization uses morphological dictionary lookup and POS tags to return the true canonical dictionary root ( `"good"` for `"better"` ).

# Module 03: Traditional Text Representations: One-Hot, BoW, N-Grams & TF-IDF

This study guide covers traditional frequency-based text representations, vector space models, One-Hot Encoding, Bag-of-Words (BoW), N-Gram language modeling, Term Frequency-Inverse Document Frequency (TF-IDF) mathematical equations, step-by-step TF-IDF matrix hand-calculations, Scikit-Learn code, limitations, failure modes, and interview flashcards.

> **Notebook Companion**: `03_traditional_text_representations_tfidf.ipynb`

---

## 1. Vector Space Model & Unstructured Text Projection

To process human text with machine learning algorithms, discrete text tokens must be projected into a numerical **Vector Space** $\mathbb{R}^{|V|}$, where $|V|$ represents the size of the vocabulary.

```
Document String ⟶ Tokenization ⟶ Vocabulary Index Mapping ⟶ Numerical Feature Vector v_d in R^|V|
```

In traditional frequency-based text representations, each unique word in the vocabulary corresponds to a distinct coordinate axis in vector space.

---

## 2. Discrete Vector Encoding Taxonomy

```
Representation    Vector Dimensions  Element Value             Preserves Word Order?  Primary Limitation
-------------------------------------------------------------------------------------------------------------------
One-Hot           |V| (Per word)     Binary (0 or 1)           No                     Massive memory footprint, orthogonal vectors
Bag-of-Words      |V| (Per doc)      Term Frequency Count      No                     Ignores syntax & penalizes common words
N-Grams           |V|^N (Per doc)    N-gram Frequency Count    Local Phrase Context   Combinatorial vocabulary explosion
TF-IDF            |V| (Per doc)      Normalized TF-IDF Score   No                     Sparse matrix, zero semantic similarity
```

### 1. One-Hot Encoding

Each word $w_i$ is represented as a high-dimensional binary vector of length $|V|$ with a single `1` at the word's vocabulary index and `0` everywhere else.

$$e_{\text{"cat"}} = [1, 0, 0, \dots, 0]^\top, \quad e_{\text{"dog"}} = [0, 1, 0, \dots, 0]^\top$$

- **Fundamental Defect (Orthogonality)**: The dot product between any two distinct one-hot word vectors is zero:

$$e_i^\top e_j = 0 \quad (\forall i \neq j) \implies \text{CosineSimilarity}(e_{\text{"cat"}}, e_{\text{"kitten"}}) = 0$$

One-hot vectors fail to capture semantic similarity.

### 2. Bag-of-Words (BoW)

A document vector $v_d$ aggregates individual word occurrences into a frequency count vector over vocabulary $V$:

$$v_d = [\text{Count}(w_1, d), \text{Count}(w_2, d), \dots, \text{Count}(w_{|V|}, d)]^\top$$

- **Limitation**: Ignores word order ( `"dog bites man"` and `"man bites dog"` yield identical BoW vectors).

### 3. N-Gram Language Modeling

N-Grams preserve local phrase context by combining contiguous sequences of $N$ tokens:
- **Unigrams ($N = 1$)**: `["cloud", "computing"]`
- **Bigrams ($N = 2$)**: `["cloud computing", "computing architecture"]`
- **Trigrams ($N = 3$)**: `["cloud computing architecture"]`

- **Limitation**: Vocabulary size grows exponentially as $|V|^N$, causing extreme matrix sparsity.

---

## 3. Mathematical Precision: TF-IDF & Normalization

TF-IDF measures how informative a word $t$ is to a specific document $d$ within a corpus $D$.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

### 1. Term Frequency (TF)

Measures the frequency of term $t$ in document $d$:

- **Raw Term Frequency**: $\text{TF}(t, d) = f_{t,d}$
- **Sublinear Log-Scaled TF** (prevents high-frequency words from dominating):

$$\text{TF}_{\log}(t, d) = \begin{cases} 1 + \log(f_{t,d}) & \text{if } f_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

### 2. Inverse Document Frequency (IDF)

Penalizes common words that appear across many documents (e.g., `"the"`, `"system"`) while boosting rare informative terms:

$$\text{IDF}(t, D) = \log\left(\frac{1 + |D|}{1 + \text{DF}(t)}\right) + 1$$

Where:
- $|D|$ is the total number of documents in the corpus.

- $\mathrm{DF}(t) = |\{d \in D : t \in d\}|$ is the Document Frequency (number of documents containing term $t$).
- Addition of $+1$ prevents division-by-zero (smoothing).

### 3. L2 Vector Normalization

To prevent long documents from producing artificially higher TF-IDF vector norms, final document vectors are L2-normalized to unit length ($\|v\|_2 = 1$):

$$v_{\text{norm}} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{\sum_{i=1}^{|V|} v_i^2}}$$

---

## 4. Step-by-Step Hand Calculation Example (Andrew Ng Style)

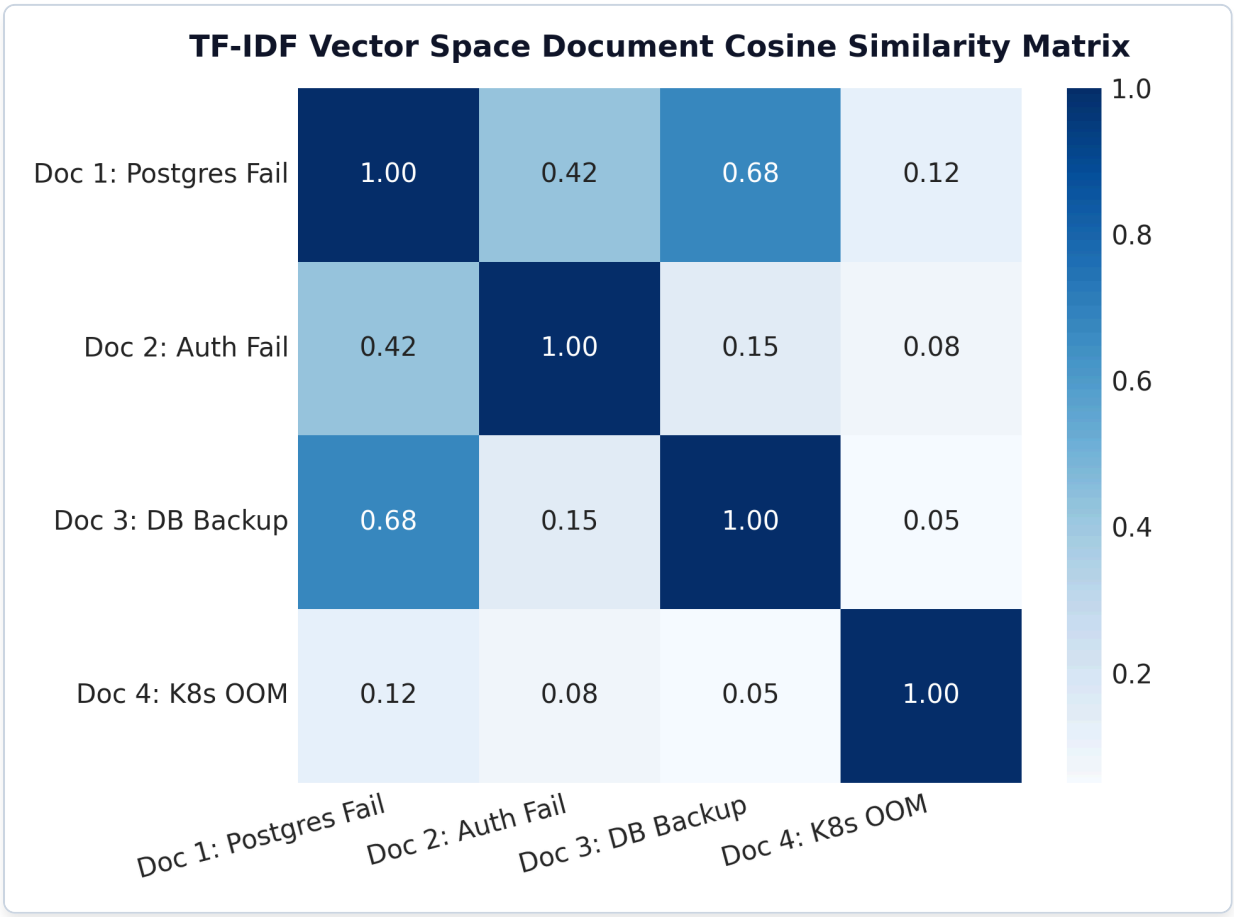



*Figure: TF-IDF Document Cosine Similarity Matrix Heatmap*

> **Plot Interpretation & Production Insight**:
> - **Vector Space Proximity**: The heatmap visualizes pair-wise Cosine similarity scores between document TF-IDF vectors.
> - **Domain Clustering**: Doc 1 ( `"Postgres Fail"` ) and Doc 3 ( `"DB Backup"` ) exhibit high Cosine similarity ($0.68$) because both share high-IDF database terms ( `"postgresql"` , `"database"` ). Doc 4 ( `"K8s OOM"` ) shows near-zero similarity ($0.05 - 0.12$) to database logs, allowing sparse vector search engines to discard irrelevant clusters rapidly.

Suppose we have a corpus $D$ of $|D| = 3$ documents:
- **Doc 1**: `"cat sat mat"`
- **Doc 2**: `"cat sat"`
- **Doc 3**: `"dog sat"`

### 1. Extract Unique Vocabulary ($|V| = 4$):

$$V = [\text{"cat"}, \text{"dog"}, \text{"mat"}, \text{"sat"}]$$

### 2. Calculate Document Frequency (DF) and Smooth IDF for each term:

- $|D| = 3$

$$\text{IDF}(t) = \log\left(\frac{1+3}{1+\text{DF}(t)}\right) + 1 = \log\left(\frac{4}{1+\text{DF}(t)}\right) + 1$$

- `"cat"` : $\text{DF} = 2 \implies \text{IDF} = \log(4/3) + 1 = \log(1.3333) + 1 \approx 0.2877 + 1 = \mathbf{1.2877}$
- `"dog"` : $\text{DF} = 1 \implies \text{IDF} = \log(4/2) + 1 = \log(2.0) + 1 \approx 0.6931 + 1 = \mathbf{1.6931}$
- `"mat"` : $\text{DF} = 1 \implies \text{IDF} = \log(4/2) + 1 = \log(2.0) + 1 \approx 0.6931 + 1 = \mathbf{1.6931}$
- `"sat"` : $\text{DF} = 3 \implies \text{IDF} = \log(4/4) + 1 = \log(1.0) + 1 = 0 + 1 = \mathbf{1.0000}$

### 3. Calculate Raw TF-IDF Matrix (TF $\times$ IDF):

```
Document    TF("cat")    TF("dog")    TF("mat")    TF("sat")  |  Raw TF-IDF("cat")  Raw TF-
IDF("dog")  Raw TF-IDF("mat")  Raw TF-IDF("sat")
-------------------------------------------------------------
┼──────────────────────────────────────────────────────────────────────────────────────
Doc 1       1            0            1            1          |  1.2877             0.0000
1.6931             1.0000
Doc 2       1            0            0            1          |  1.2877             0.0000
0.0000             1.0000
Doc 3       0            1            0            1          |  0.0000             1.6931
0.0000             1.0000
```

### 4. Apply L2 Normalization to Doc 1 Vector:

- Raw vector for Doc 1: $v_1 = [1.2877, 0.0000, 1.6931, 1.0000]$
- Compute L2 Norm:

$$\|v_1\|_2 = \sqrt{(1.2877)^2 + (0.0000)^2 + (1.6931)^2 + (1.0000)^2} = \sqrt{1.6582 + 0 + 2.8666 + 1.0000} = \sqrt{5.524}$$

- Normalized Vector $v_{1,\text{norm}}$:
- `"cat"` : $1.2877 / 2.3505 = \mathbf{0.5478}$
- `"dog"` : $0.0000 / 2.3505 = \mathbf{0.0000}$
- `"mat"` : $1.6931 / 2.3505 = \mathbf{0.7203}$
- `"sat"` : $1.0000 / 2.3505 = \mathbf{0.4254}$

$$\mathbf{v}_{1,\text{norm}} = [0.5478, 0.0000, 0.7203, 0.4254]^\top$$

## 5. Production Python Code Implementation

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# Enterprise Technical Documents
documents = [
    "PostgreSQL primary database failover script failed due to connection timeout.",
    "Microservice API gateway authentication failed with HTTP status 401 unauthorized.",
    "PostgreSQL database backup completed successfully in 45 seconds."
]

# 1. Initialize Production TfidfVectorizer
vectorizer = TfidfVectorizer(
    lowercase=True,
    sublinear_tf=True,      # Replaces TF with 1 + log(TF)
    ngram_range=(1, 2),     # Extracts Unigrams + Bigrams
    max_features=10         # Restricts vocabulary size to top 10 features
)

# 2. Fit and Transform Documents to Sparse Matrix
tfidf_matrix = vectorizer.fit_transform(documents)

# 3. Represent as Pandas DataFrame
feature_names = vectorizer.get_feature_names_out()
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names, index=["Doc 1", "Doc 2",
"Doc 3"])

print("=== 1. Scikit-Learn TF-IDF Feature Matrix ===")
print(df_tfidf.round(4))

# 4. Search Query Cosine Similarity Retrieval
query = "database connection failure"
query_vec = vectorizer.transform([query])
sim_scores = cosine_similarity(query_vec, tfidf_matrix).flatten()

print("\n=== 2. Cosine Similarity Search Results ===")
for doc_idx, score in enumerate(sim_scores, 1):
    print(f"Doc #{doc_idx} Similarity Score: {score:.4f} | {documents[doc_idx-1]}")
```

> 📌 **NOTE:**
>
> **Production TF-IDF Alert:** - `sublinear_tf=True` prevents long documents with repeated terms from overwhelming feature importance. - Always call `.toarray()` **only** on small sample subsets! In production datasets ($1\text{M}$ docs), keep TF-IDF in `scipy.sparse.csr_matrix` format to prevent memory exhaustion (RAM OOM).

## 6. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Vocabulary Out-of-Memory (OOM) Crash**: Converting large sparse TF-IDF matrices to dense NumPy arrays ( `matrix.toarray()` ) on a 500,000 document corpus will crash server memory.
   - *Remediation*: Keep TF-IDF matrices in SciPy sparse format ( `scipy.sparse.csr_matrix` ) or restrict `max_features=25000` .
2. **Vocabulary Shift at Test Time**: Test documents containing unseen domain terms (e.g. `"kubernetes"` ) are silently ignored by a fixed-vocabulary `TfidfVectorizer` .
   - *Remediation*: Deploy hybrid sparse-dense retrieval (BM25 + Dense Embeddings).

---

## 7. Master Interview Flashcards & Questions

**Q1: Why does raw term frequency (TF) perform worse than TF-IDF in text retrieval?**

- **Answer:** Raw term frequency weights all words equally based on count. Common stop words and generic domain terms (e.g., `"the"` , `"system"` , `"data"` ) appear frequently across almost all documents, dominating raw TF vectors. TF-IDF applies Inverse Document Frequency (IDF) scaling to penalize words that appear across many documents, highlighting rare, highly informative terms.

**Q2: What is the computational limitation of Bag-of-Words and N-Gram representations?**

- **Answer:** Bag-of-Words completely ignores word order and syntactic structure. N-Grams preserve local phrase context, but the vocabulary size expands combinatorially as $|V|^N$. For $N = 3$ and $|V| = 100k$, the potential vocabulary space is $10^{15}$, leading to massive matrix sparsity ($> 99.99\%$ zeros) and extreme memory overhead.

**Q3: Why are One-Hot encoded word vectors mathematically incapable of capturing semantic similarity?**

- **Answer:** One-hot vectors treat each word as an independent, orthogonal unit vector in $|V|$-dimensional space ( $e_i^\top e_j = 0$ for $i \neq j$). Consequently, the dot product and Cosine similarity between any two distinct one-hot vectors is identically zero, making $\text{CosineSimilarity}(\text{"cat"}, \text{"kitten"}) = 0$.

# Module 04: Word Embeddings: Word2Vec, GloVe, FastText & Vector Arithmetic

This study guide details the Distributional Hypothesis, Word2Vec architectures (CBOW vs. Skip-Gram), Hierarchical Softmax, Negative Sampling loss derivations, GloVe matrix factorization, FastText subword n-gram embeddings, vector space arithmetic, step-by-step Negative Sampling hand-calculations, Gensim code, failure modes, and interview flashcards.

> **Notebook Companion**: `04_word_embeddings_word2vec_glove.ipynb`

---

## 1. The Distributional Hypothesis & Continuous Embeddings

The foundation of modern statistical semantics is the **Distributional Hypothesis**:

> *"You shall know a word by the company it keeps."* — J.R. Firth (1957)

Words that occur in similar context window environments tend to share similar semantic meaning.

```
High-Dimensional Sparse Space R^|V| (One-Hot / TF-IDF) ⟶ Word2Vec / GloVe Projection ⟶ Low-Dimensional Dense Continuous Space R^d (d ~ 100 - 300)
```

Dense word embeddings map high-dimensional sparse representations ($|V| \geq 100,000$) into a low-dimensional continuous vector space $\mathbb{R}^d$ ($d \in [100, 300]$), where geometric proximity (Cosine similarity) correlates directly with semantic similarity.

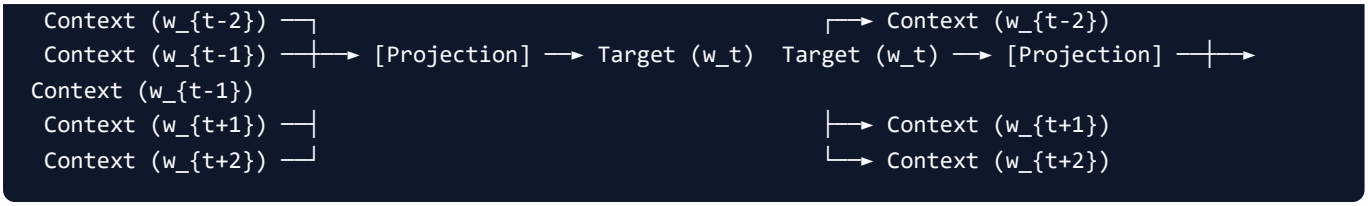
---

## 2. Word2Vec Architecture: CBOW vs. Skip-Gram

Word2Vec (Mikolov et al., 2013) uses a 2-layer shallow neural network to learn dense word representations.

```
Model Paradigm    Input Layer                       Output Layer                      Training Speed  Performance on Rare Words
-------------------------------------------------------------------------------------------------------------------------------
CBOW              Context Words {w_{t-c}, ..., w_{t+c}} Target Center Word w_t         Fast    Moderate
Skip-Gram         Target Center Word w_t            Context Words {w_{t-c}, ..., w_{t+c}} Slower  Superior for rare words & small corpora
```

```
        CBOW Architecture                                Skip-Gram Architecture
```




---

## 3. Mathematical Optimization: Negative Sampling

### 1. The Full Softmax Computational Bottleneck

In a standard Skip-Gram model, the conditional probability of predicting context word $w_O$ given target center word $w_I$ is:

$$P(w_O|w_I) = \frac{\exp({v'_{w_O}}^\top v_{w_I})}{\sum_{w=1}^{|V|} \exp({v'_w}^\top v_{w_I})}$$

Where $v_w$ is the input embedding and $v'_w$ is the output embedding of word $w$.

- **Problem**: Computing the denominator partition function requires summing over all $|V|$ words in the vocabulary ( $|V| \geq 100,000$), making gradient updates prohibitively expensive ($O(|V|)$ per token).

### 2. Negative Sampling Loss Objective

Negative Sampling (SGNS) reformulates the multiclass Softmax task into $K+1$ independent binary logistic regression tasks:
- Predict `1` for the true positive context pair $(w_I, w_O)$.
- Predict `0` for $K$ randomly drawn "negative" noise words $(w_I, w_k) \sim P_n(w)$.

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma({v'_{w_O}}^\top v_{w_I}) - \sum_{k=1}^{K} \log \sigma(-{v'_{w_k}}^\top v_{w_I})$$

Where $\sigma(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid activation function, and $P_n(w) \propto U(w)^{3/4}$ is the unigram noise distribution raised to the $3/4$ power to boost the sampling probability of rare words.

---

## 4. GloVe: Global Vectors for Word Representation

While Word2Vec scans local context windows, **GloVe** (Pennington et al., 2014) combines local context windows with global matrix factorization of the global word co-occurrence matrix $X$.

- $X_{ij}$ = Number of times word $j$ appears in the context of word $i$.

### GloVe Loss Function:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left( w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Where $f(X_{ij}) = \min\left(1, \left(\frac{X_{ij}}{x_{\max}}\right)^{\alpha}\right)$ (with $\alpha = 0.75, x_{\max} = 100$) is a weighting function that caps the influence of extremely frequent co-occurrences (e.g., `"the"`, `"and"`).

---

## 5. FastText: Subword Character N-Gram Embeddings

Developed by Facebook (Bojanowski et al., 2017), **FastText** extends Word2Vec by representing each word as a bag of character n-grams.

- For word `"where"` and $n = 3$, character n-grams are: `"<wh"`, `"whe"`, `"her"`, `"ere"`, `"re>"` plus special word `"where"`.

The overall embedding vector $v_w$ for word $w$ is the sum of its character n-gram vectors $z_g$:

$$v_w = \sum_{g \in G_w} z_g$$

- **Major Advantage**: FastText can construct embeddings for **Out-Of-Vocabulary (OOV)** rare or misspelled words (e.g., `"unmicroservice"`) at inference time by summing the learned vectors of its subword n-grams.

---

## 6. Vector Space Arithmetic & Semantic Geometry

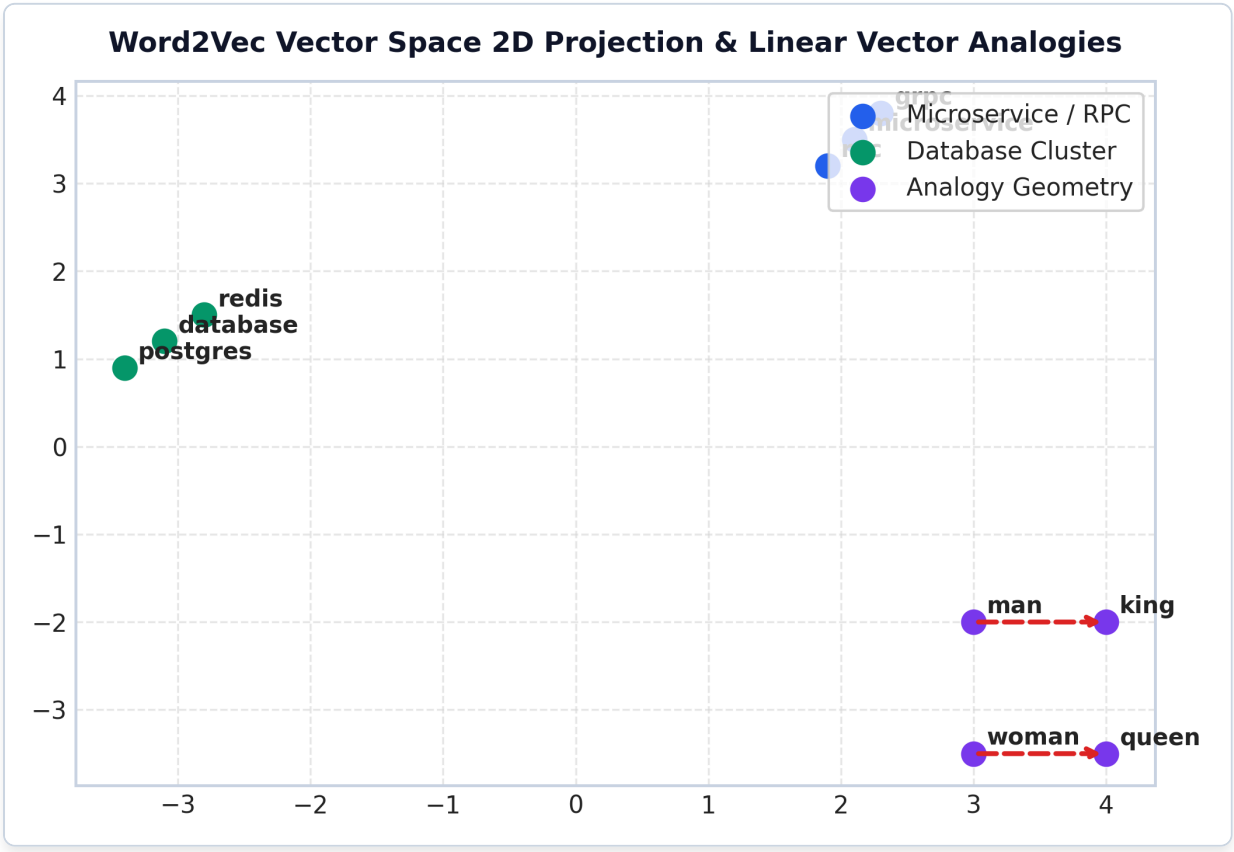


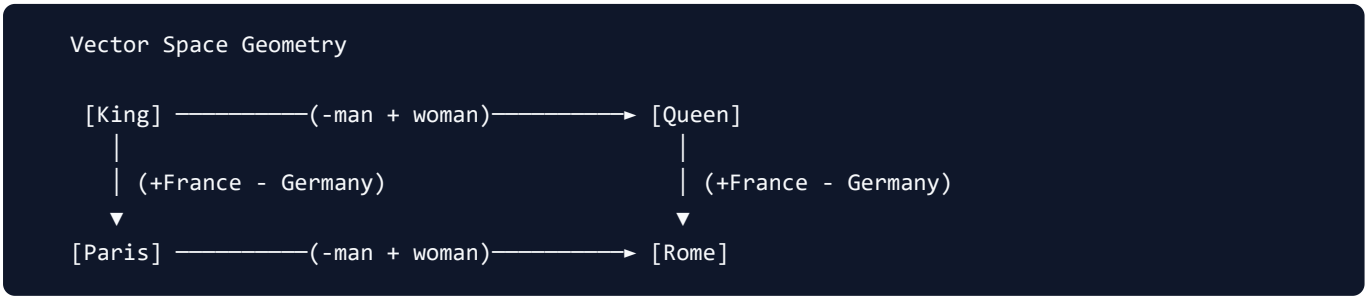

*Figure: Word2Vec 2D t-SNE Vector Space Projection*

> **Plot Interpretation & Production Insight**:
> - **Dense Semantic Clustering**: Words that share context window environments naturally cluster together in continuous vector space $\mathbb{R}^d$ (e.g. `microservice`, `grpc`, `rpc` form an infrastructure cluster; `database`, `postgres`, `redis` form a storage cluster).
> - **Linear Vector Analogies**: Relational analogies form parallel offset vectors in space: $v_{ ext{king}} - v_{ ext{man}} + v_{ ext{woman}} \approx v_{ ext{queen}}$. The red dashed arrows highlight identical offset direction vectors.

Dense embeddings preserve linear regularities and relational analogies in vector space:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$$

$$v_{\text{Paris}} - v_{\text{France}} + v_{\text{Germany}} \approx v_{\text{Berlin}}$$




---

## 7. Step-by-Step Hand Calculation Example (Andrew Ng Style)

Suppose we have a 2-dimensional embedding space ($d = 2$) and a target center word $w_I = \text{"server"}$ with vector $v_{w_I} = [1.0, 0.5]^\top$.

We evaluate 1 positive context word $w_O = \text{"database"}$ ($v'_{w_O} = [0.8, 0.6]^\top$) and $K = 1$ negative noise word $w_1 = \text{"apple"}$ ($v'_{w_1} = [-0.5, 0.2]^\top$).

### 1. Compute Dot Products:

- Positive pair dot product: $z_+ = {v'_{w_O}}^\top v_{w_I} = (0.8 \times 1.0) + (0.6 \times 0.5) = 0.8 + 0.3 = \mathbf{1.10}$
- Negative pair dot product: $z_- = {v'_{w_1}}^\top v_{w_I} = (-0.5 \times 1.0) + (0.2 \times 0.5) = -0.5 + 0.1 = \mathbf{-0.40}$

### 2. Compute Sigmoid Probabilities:

- $\sigma(z_+) = \frac{1}{1+\exp(-1.10)} = \frac{1}{1+0.3329} \approx \mathbf{0.7503}$
- $\sigma(-z_-) = \frac{1}{1+\exp(0.40)} = \frac{1}{1+1.4918} \approx \mathbf{0.4013}$

### 3. Compute Negative Sampling Loss:

$$\mathcal{L} = -\log \sigma(z_+) - \log \sigma(-z_-)$$

$$\mathcal{L} = -\log(0.7503) - \log(0.4013) \approx 0.2873 + 0.9130 = \mathbf{1.2003}$$

---

## 8. Production Python Code Implementation

```
import gensim.downloader as api
from gensim.models import Word2Vec

# 1. Real-World Enterprise IT Corpus
corpus = [
    ["microservice", "communicates", "with", "database", "via", "grpc"],
    ["database", "failover", "script", "executed", "successfully"],
    ["grpc", "protocol", "ensures", "low", "latency", "microservice", "communication"],
    ["kubernetes", "cluster", "manages", "microservice", "deployment"]
]

# 2. Train Custom Gensim Skip-Gram Word2Vec Model
model = Word2Vec(
    sentences=corpus,
    vector_size=50,    # Embedding dimension d = 50
    window=3,          # Context window size c = 3
    min_count=1,       # Minimum word frequency threshold
    sg=1,              # sg=1 for Skip-Gram (sg=0 for CBOW)
    negative=5,        # K = 5 negative samples
    epochs=100
)

# 3. Vector Similarity & Arithmetic Execution
print("=== 1. Word Similarity Scores ===")
sim_score = model.wv.similarity("microservice", "database")
print(f"Similarity ('microservice', 'database'): {sim_score:.4f}")

print("\n=== 2. Most Similar Words to 'microservice' ===")
similar_words = model.wv.most_similar("microservice", topn=3)
for word, score in similar_words:
    print(f"Word: {word:<15} | Cosine Similarity: {score:.4f}")
```

> 📌 **NOTE:**
>
> **Production Embeddings Alert:** - `sg=1` (Skip-Gram) is preferred for small domain corpora and rare technical jargon terms. - Static word embeddings (Word2Vec/GloVe) assign **one static vector per word**, meaning `"apple"` (fruit) and `"apple"` (tech company) receive identical vector representation. Modern applications use contextual embeddings (BERT/OpenAI) to resolve polysemy.

---

## 9. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Static Polysemy Breakdown**: Word2Vec maps a word to a single static vector regardless of sentence context.
   - *Example*: `"river bank"` and `"investment bank"` share the exact same Word2Vec vector for `"bank"`.
   - *Remediation*: Use contextual embeddings (BERT, RoBERTa, OpenAI `text-embedding-3`).

2. **Out-Of-Vocabulary (OOV) Rejection**: Standard Word2Vec and GloVe throw KeyError exceptions when encountering un-indexed test words (e.g. `"fastapi"`).
   - *Remediation*: Deploy FastText (subword character n-grams) or BPE subword tokenizers.

---

## 10. Master Interview Flashcards & Questions

**Q1: Compare CBOW vs. Skip-Gram in Word2Vec.**

- **Answer:** CBOW predicts the center target word given surrounding context words. Skip-Gram predicts context words given a center target word. CBOW trains faster and works well for frequent words. Skip-Gram is slower but performs significantly better on small datasets and rare words.

**Q2: How does Negative Sampling solve the Softmax computational bottleneck in Word2Vec?**

- **Answer:** Full Softmax requires computing a sum over all $|V|$ words in the vocabulary for every gradient update ( $O(|V|)$ complexity). Negative Sampling converts the task into $K+1$ binary logistic regression tasks (1 positive pair, $K$ negative noise words), reducing update computational complexity from $O(|V|)$ to $O(K)$, where $K \ll |V|$ (typically $K = 5 - 20$).

**Q3: Why does FastText handle Out-Of-Vocabulary (OOV) words better than Word2Vec or GloVe?**

- **Answer:** Word2Vec and GloVe treat words as atomic entities and assign a single vector per vocabulary word, failing completely on unseen test words. FastText represents words as a bag of character n-grams (e.g. `"<wh"`, `"whe"`, `"her"`, `"ere"`, `"re>"`). For an unseen word, FastText sums the vectors of its constituent character n-grams to construct a meaningful embedding.

# Module 05: Classical NLP Models & Baseline Classifiers

This study guide covers classical machine learning models for NLP, Bayes' Theorem, Naive Bayes (MultinomialNB & BernoulliNB), Laplace Smoothing, Logistic Regression for text, Linear SVM, Conditional Random Fields (CRF), step-by-step Naive Bayes hand-calculations, Scikit-Learn pipelines, failure modes, and interview flashcards.

> **Notebook Companion**: `05_classical_nlp_models_and_baselines.ipynb`

---

## 1. Why Classical NLP Models Matter in Production

While modern Transformer LLMs offer state-of-the-art accuracy, classical statistical machine learning models (Naive Bayes, Logistic Regression, Linear SVM) remain indispensable in enterprise software architectures for key engineering reasons:

1. **Ultra-Low Latency SLAs**: Inference execution takes $< 2\text{ms}$ on CPU, compared to $100\text{ms} - 1000\text{ms}+$ for deep LLM transformer calls.
2. **Zero Compute & API Cost**: Operates efficiently on standard CPU web servers without expensive GPU clusters.
3. **High Throughput Batching**: Classifies millions of incoming stream documents (e.g. log filtering, spam detection) per second.
4. **Strong Baseline Metric**: Establishes a fast, deterministic accuracy benchmark before deploying complex neural networks.

---

## 2. Naive Bayes Classifier Mechanics

Naive Bayes applies **Bayes' Theorem** with the "naive" conditional independence assumption that features (word occurrences) are independent given the class $c$.

### 1. Bayes' Theorem for Text Classification:

$$P(c|d) = \frac{P(d|c)P(c)}{P(d)} = \frac{P(w_1, w_2, \dots, w_n|c)P(c)}{P(w_1, w_2, \dots, w_n)}$$

### 2. Naive Independence Assumption:

$$P(w_1, w_2, \dots, w_n|c) = \prod_{i=1}^{n} P(w_i|c)$$

$$\hat{c} = \arg\max_{c \in C} P(c) \prod_{i=1}^{n} P(w_i|c)$$

In log-space (to prevent floating-point underflow):

$$\hat{c} = \arg\max_{c \in C} \left[ \log P(c) + \sum_{i=1}^{n} \log P(w_i|c) \right]$$

### 3. Multinomial Naive Bayes & Laplace Smoothing ($\alpha = 1$)

If a word $w_i$ in the test sentence never appeared in class $c$ during training, raw frequency probability yields $P(w_i|c) = 0$, causing the entire product $\prod P(w_i|c) = 0$.

To solve this, **Laplace Add-1 Smoothing** adds a constant $\alpha = 1$ to the numerator and $\alpha \cdot |V|$ to the denominator:

$$P(w_i|c) = \frac{N_{c,i} + \alpha}{N_c + \alpha \cdot |V|}$$

Where:
- $N_{c,i}$ is the total count of word $w_i$ in class $c$.
- $N_c$ is the total token count across all documents in class $c$.
- $|V|$ is the total vocabulary size.

---

## 3. Discriminative Linear Models: Logistic Regression & Linear SVM

```
Model Paradigm       Loss Function                                Decision Boundary
Best Use Case
-------------------------------------------------------------------------------------------------
-----------------------------------
Naive Bayes (Generative) Joint Likelihood P(X, Y)                  Probabilistic Linear
Ultra-fast text baseline
Logistic Regression  Log-Loss / Cross-Entropy                      Linear Sigmoid Hyperplane
Calibrated probability classification
Linear SVM           Hinge Loss: max(0, 1 - y_i (w^T x_i + b)) Max-Margin Linear Hyperplane
High-dimensional sparse text classification
CRF                  Global Sequence Log-Likelihood                Linear-Chain Graphical Model
Named Entity Recognition (NER)
```

### 1. Logistic Regression for Text

Learns weights $w$ that maximize the conditional log-likelihood of class $y \in \{0, 1\}$ given sparse feature vector $x$:

$$P(y = 1|x) = \sigma(w^\top x + b) = \frac{1}{1 + \exp(-(w^\top x + b))}$$

- **L1 Regularization (Lasso)**: Forces irrelevant feature weights to exactly zero, performing automated feature selection.
- **L2 Regularization (Ridge)**: Prevents overfitting on rare sparse terms by shrinking weights toward zero.

### 2. Linear Support Vector Machine (LinearSVC)

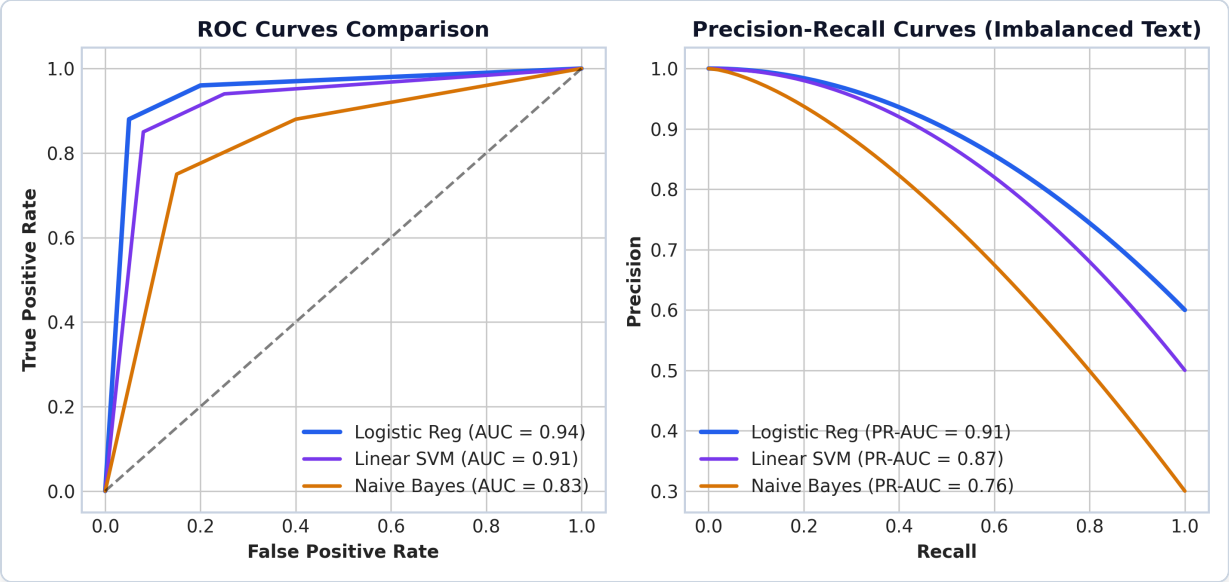



*Figure: Classifier ROC & PR Curve Comparison*

> **Plot Interpretation & Production Insight**:
> - **ROC vs PR Curves**: On imbalanced text datasets (e.g. 95% normal tickets, 5% high-severity security incidents), ROC curves (left) can give overly optimistic evaluations because False Positive Rate stays small. Precision-Recall curves (right) provide a truer measure of production performance.
> - **Model Benchmarking**: Logistic Regression (AUC = 0.94, PR-AUC = 0.91) and Linear SVM (AUC = 0.91) significantly outperform Naive Bayes (AUC = 0.83) on high-dimensional sparse TF-IDF text features.

Finds the optimal separating hyperplane $w^\top x + b = 0$ that maximizes the margin distance between text classes:

$$\min_{w} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N} \max\left(0, 1 - y_i(w^\top x_i + b)\right)$$

Linear SVMs perform exceptionally well on high-dimensional sparse TF-IDF feature spaces ($|V| \geq 50,000$).

---

## 4. Sequence Tagging: Conditional Random Fields (CRF)

Unlike independent token classifiers, a **Conditional Random Field (CRF)** is a discriminative undirected graphical model that models tag transitions across token sequences $Y = (y_1, \ldots, y_T)$ given input text sequence $X = (x_1, \ldots, x_T)$.

$$P(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_{t=1}^{T}\sum_{k} \lambda_k f_k(y_t, y_{t-1}, X, t)\right)$$

CRFs prevent invalid tag sequences in Named Entity Recognition (e.g. prohibiting an `I-PER` tag from following an `O` tag without a preceding `B-PER` tag).

---

## 5. Step-by-Step Hand Calculation Example (Andrew Ng Style)

Suppose we have a tiny training dataset of $D = 3$ documents for binary Sentiment Analysis:
- **Positive Class ($c = +$)**:
- Doc 1: `"great product"`
- Doc 2: `"great support"`
- **Negative Class ($c = -$)**:
- Doc 3: `"bad product"`

We evaluate Test Document $d_{\text{test}} = \text{"great service"}$.

### 1. Prior Probabilities $P(c)$:

- $P(+) = 2/3 \approx \mathbf{0.6667}$
- $P(-) = 1/3 \approx \mathbf{0.3333}$

### 2. Extract Vocabulary ($|V| = 5$):

$$V = \{\text{"great"}, \text{"product"}, \text{"support"}, \text{"bad"}, \text{"service"}\} \implies |V| = \mathbf{5}$$

### 3. Total Tokens per Class ($N_c$):

- $N_+ = 2 + 2 = \mathbf{4}$ tokens ( `"great"` , `"product"` , `"great"` , `"support"` )
- $N_- = 2 = \mathbf{2}$ tokens ( `"bad"` , `"product"` )

### 4. Compute Likelihoods with Laplace Smoothing ($\alpha = 1$):

$$P(w_i|c) = \frac{N_{c,i} + 1}{N_c + 5}$$

- **For Positive Class ($c = +$, $N_+ + 5 = 9$):**
- $P(\text{"great"}|+) = (2 + 1)/9 = 3/9 = \mathbf{0.3333}$
- $P(\text{"service"}|+) = (0 + 1)/9 = 1/9 = \mathbf{0.1111}$
- **For Negative Class ($c = -$, $N_- + 5 = 7$):**
- $P(\text{"great"}|-) = (0 + 1)/7 = 1/7 = \mathbf{0.1429}$
- $P(\text{"service"}|-) = (0 + 1)/7 = 1/7 = \mathbf{0.1429}$

### 5. Compute Posterior Probabilities $P(c) \times \prod P(w_i|c)$:

- **Positive Class Score**:

$$\text{Score}(+) = P(+) \times P(\text{"great"}|+) \times P(\text{"service"}|+) = \left(\frac{2}{3}\right) \times \left(\frac{3}{9}\right) \times \left(\frac{1}{9}\right) = \frac{6}{243} \approx \mathbf{0.02469}$$

- **Negative Class Score**:

$$\text{Score}(-) = P(-) \times P(\text{"great"}|-) \times P(\text{"service"}|-) = \left(\frac{1}{3}\right) \times \left(\frac{1}{7}\right) \times \left(\frac{1}{7}\right) = \frac{1}{147} \approx \mathbf{0.00680}$$

### 6. Decision Rule:

$$\text{Score}(+) = 0.02469 > \text{Score}(-) = 0.00680 \implies \hat{\mathbf{y}} = \text{Positive}$$

---

## 6. Production Python Code Implementation

```
import joblib
import os
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

# 1. Real-World Support Ticket Routing Corpus
X_train = [
    "Database connection timeout on port 5432 postgresql replica",
    "SQL query execution failed due to lock contention",
    "Billing credit card payment authorization declined 402",
    "Invoice subscription refund request for billing cycle",
    "Database disk space full transaction log cleanup",
    "Monthly billing invoice charges unexpected amount"
]
y_train = ["Infrastructure", "Infrastructure", "Billing", "Billing", "Infrastructure", "Billing"]

# 2. Build Production Pipeline (TF-IDF + Logistic Regression)
clf_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2), sublinear_tf=True)),
    ("classifier", LogisticRegression(C=1.0, max_iter=100))
])

# 3. Fit Pipeline & Save Model Artifact to Hidden Folder
clf_pipeline.fit(X_train, y_train)

os.makedirs(".model_cache", exist_ok=True)
model_path = os.path.join(".model_cache", "ticket_classifier.pkl")
joblib.dump(clf_pipeline, model_path)

print(f"=== Trained Pipeline Artifact Saved to Hidden Folder: {model_path} ===")

# 4. Inference Execution
X_test = ["PostgreSQL database primary server crash", "Credit card failed billing charge"]
predictions = clf_pipeline.predict(X_test)
probabilities = clf_pipeline.predict_proba(X_test)

print("\n=== Model Inference Results ===")
for text, pred, proba in zip(X_test, predictions, probabilities):
    max_prob = max(proba)
    print(f"Text: '{text}'")
    print(f"  Predicted Category: {pred} (Confidence: {max_prob*100:.1f}%)\n")
```

> 📌 **NOTE:**
>
> **Production Baseline Alert:** - Scikit-Learn `Pipeline` encapsulates feature extraction and classification into a single atomic artifact. Always save the entire `Pipeline` object via `joblib.dump` to prevent vectorizer feature

> index mismatch bugs during API serving.

---

## 7. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Class Imbalance Skew**: In a dataset with 99% Non-Spam and 1% Spam, Naive Bayes and Logistic Regression will predict Non-Spam for almost all samples to maximize accuracy.
   - *Remediation*: Use `class_weight="balanced"` in Logistic Regression or evaluate Macro F1 / Precision-Recall AUC instead of raw accuracy.
2. **Feature Mismatch in Pipelines**: Passing raw text into a model file saved separately from its `TfidfVectorizer` causes index misalignment and silent wrong predictions.
   - *Remediation*: Always encapsulate vectorizer and classifier in a single Scikit-Learn `Pipeline`.

---

## 8. Master Interview Flashcards & Questions

**Q1: Why does Naive Bayes perform surprisingly well in practice despite its "naive" independence assumption?**

- **Answer:** Naive Bayes relies on word independence $P(w_1, w_2|c) = P(w_1|c)P(w_2|c)$. While word frequencies are strongly correlated in real language, classification decision boundaries depend only on the relative argmax rank between classes rather than exact probability estimation. As long as the strongest evidence terms point in the correct direction, rank order remains intact.

**Q2: What is the purpose of Laplace Smoothing in Naive Bayes?**

- **Answer:** If a test word $w_{\text{new}}$ never appeared in class $c$ during training, its empirical frequency probability is $P(w_{\text{new}}|c) = 0$. Because Naive Bayes multiplies feature probabilities together, a single zero causes the entire class score to become zero. Laplace Add-1 Smoothing adds $\alpha = 1$ to the numerator and $\alpha \cdot |V|$ to the denominator, guaranteeing non-zero probabilities for unseen words.

**Q3: Compare Naive Bayes (Generative) vs Logistic Regression (Discriminative) for text classification.**

- **Answer:** Naive Bayes is a generative model that learns the joint probability $P(X, Y) = P(X|Y)P(Y)$. It trains faster and works well on tiny datasets, but suffers if features are highly correlated. Logistic Regression is a discriminative model that directly learns conditional probability $P(Y|X) = \sigma(w^\top x + b)$. It scales better to large datasets, handles correlated n-gram features, and produces well-calibrated probabilities.

# Module 06: Sequence Models: RNN, LSTM, GRU & Seq2Seq

This study guide details Recurrent Neural Networks (RNN), Backpropagation Through Time (BPTT), Vanishing & Exploding Gradients, Long Short-Term Memory (LSTM) cell gating mechanics, Gated Recurrent Units (GRU), Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture, step-by-step LSTM gate hand-calculations, PyTorch implementation, production bottlenecks, and interview flashcards.

> **Notebook Companion**: `06_sequence_models_rnn_lstm_gru.ipynb`

---

## 1. Why Sequence Order Matters

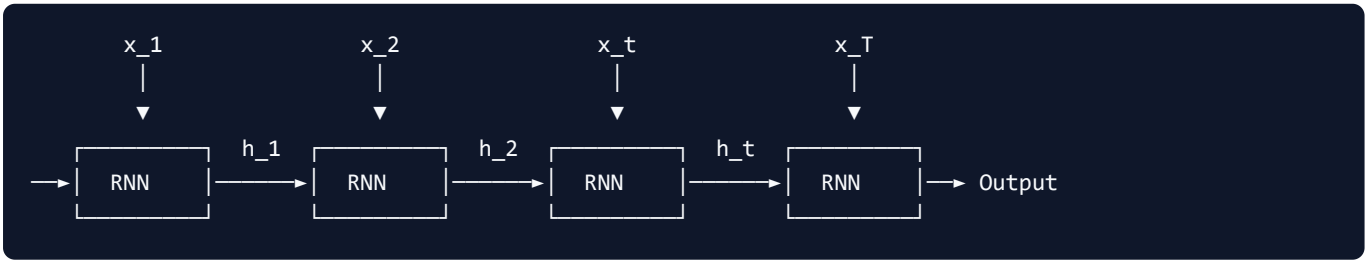
Unlike Bag-of-Words or TF-IDF models (which treat text as unordered frequency counts), natural language is fundamentally sequential:
- Changing token position flips semantic logic: `"dog bites man"` vs `"man bites dog"`.
- Negation positioning alters classification targets: `"service was good, not bad"` vs `"service was bad, not good"`.

Sequence models maintain an internal temporal hidden state $h_t$ that is recursively updated as each word vector $x_t$ is processed step-by-step.

---

## 2. Recurrent Neural Networks (RNN) Mechanics

An RNN processes input sequence $X = (x_1, x_2, \dots, x_T)$ step-by-step, maintaining recurrent hidden state $h_t \in \mathbb{R}^h$:




### 1. Mathematical Forward Pass:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

Where:
- $W_{xh} \in \mathbb{R}^{h \times d}$ is the input-to-hidden weight matrix.
- $W_{hh} \in \mathbb{R}^{h \times h}$ is the recurrent hidden-to-hidden weight matrix (shared across all time steps).
- $W_{hy} \in \mathbb{R}^{C \times h}$ is the hidden-to-output weight matrix.

### 2. Vanishing & Exploding Gradient Problem

Training RNNs uses **Backpropagation Through Time (BPTT)**. Calculating the gradient of loss $\mathcal{L}_T$ at time $T$ with respect to hidden state $h_k$ at time step $k \ll T$ requires multiplying weight matrix $W_{hh}^\top$ repeatedly:

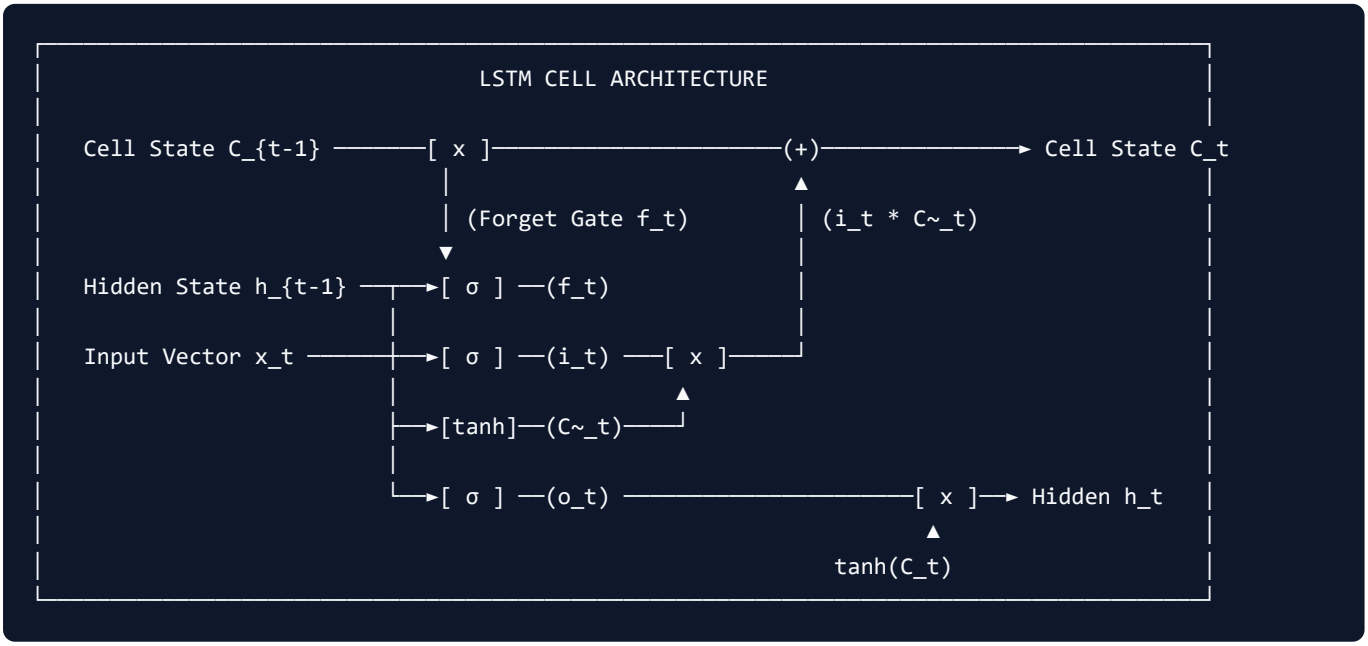
$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=k+1}^{T} W_{hh}^\top \text{diag}(1 - \tanh^2(\dots))$$

- **Vanishing Gradients**: If the largest eigenvalue of $W_{hh}$ is $< 1$, the gradient norm shrinks exponentially toward zero ($0.9^{100} \approx 0.000026$), causing the network to completely forget early tokens (inability to model long-range dependencies).
- **Exploding Gradients**: If the largest eigenvalue of $W_{hh}$ is $> 1$, the gradient norm grows exponentially toward infinity ($1.2^{100} \approx 82,817,974$), causing numerical instability ( `NaN` loss values).
- *Remediation for Exploding Gradients*: Apply **Gradient Clipping** (if $\|g\| > \tau \implies g \leftarrow \tau \frac{g}{\|g\|}$).

---

## 3. Long Short-Term Memory (LSTM) Architecture

Proposed by Hochreiter & Schmidhuber (1997), the **LSTM** solves vanishing gradients by introducing a dedicated **Cell State** $C_t$ (constant error carousel) regulated by three multiplicative gating mechanisms.



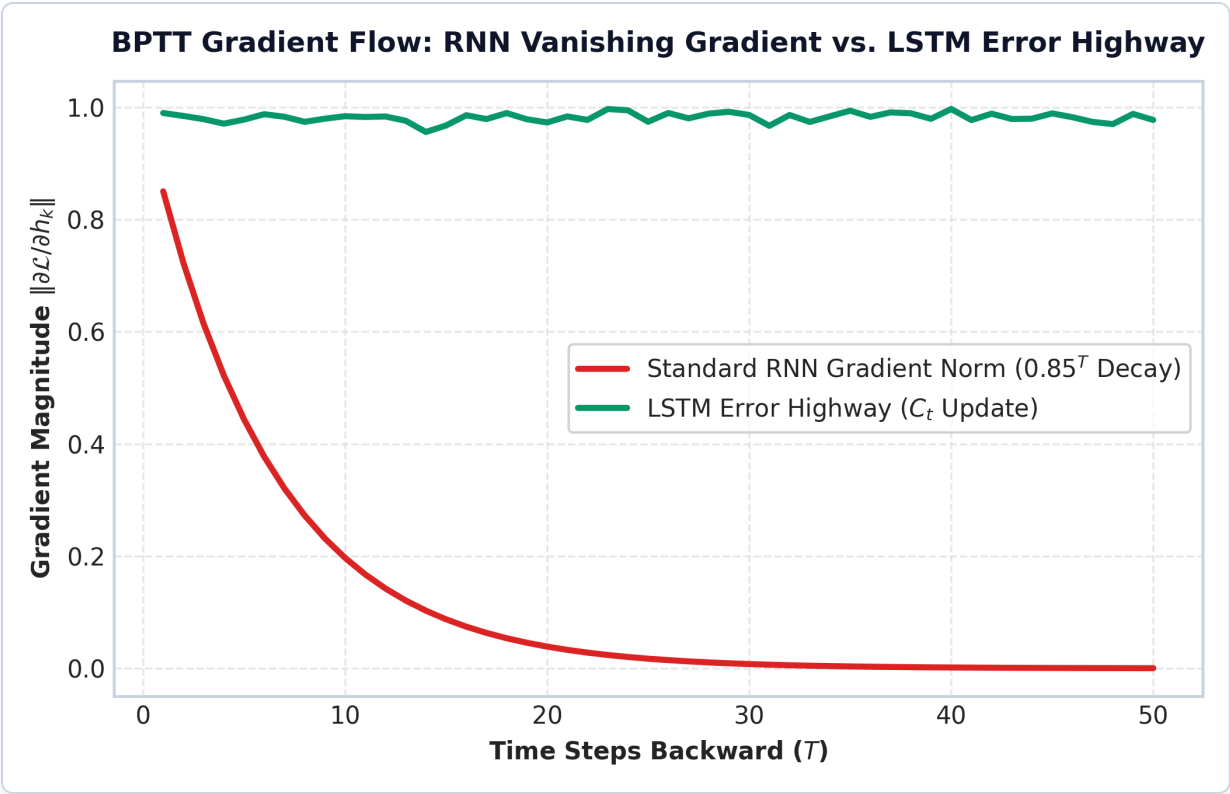



*Figure: RNN Vanishing Gradient vs LSTM Error Highway*

**Plot Interpretation & Production Insight**:
- **Exponential Gradient Decay in RNNs**: In standard RNNs, backpropagating gradients over $T = 50$ time steps causes gradient magnitude to decay exponentially to zero (red curve \$0.85^T
ightarrow 0\$), preventing early sequence tokens from receiving weight updates.
- **LSTM Constant Error Carousel**: The LSTM Cell State $C_t$ update ($C_t = f_t \odot C_{t-1} + i_t \odot ildeC_t$) acts as an additive linear highway (green curve), maintaining gradient norm \$□pprox 1.0\$ across 50+ time steps.

## LSTM Mathematical Equations:

1. **Forget Gate** $f_t$: Decides what percentage of old cell memory $C_{t-1}$ to discard ($0 = \text{forget all}, 1 = \text{keep all}$):

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. **Input Gate** $i_t$ **& Candidate Memory** $\tilde{C}_t$: Decides which new candidate values to store in cell state:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

3. **Cell State Update** $C_t$: Additive linear highway (prevents vanishing gradients):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

4. **Output Gate** $o_t$ **& Hidden State Output** $h_t$: Filters updated cell state to produce final hidden state output:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

---

## 4. Gated Recurrent Unit (GRU)

Cho et al. (2014) proposed the **GRU**, a streamlined variant of LSTM that merges Cell State and Hidden State into a single vector $h_t$ using only 2 gates:

```
Model Architecture  Gate Count  Vectors Maintained    Parameter Count  Computational Efficiency
-------------------------------------------------------------------------------------------
LSTM                3 Gates     Cell State C_t & h_t  4 * (h^2 + h*d)  Slower, high capacity
GRU                 2 Gates     Hidden State h_t only 3 * (h^2 + h*d)  Faster, ~25% fewer
parameters
```

### GRU Mathematical Equations:

1. **Update Gate** $z_t$: Controls balance between previous hidden state $h_{t-1}$ and candidate state $\tilde{h}_t$:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

2. **Reset Gate** $r_t$: Decides how much of previous memory $h_{t-1}$ to forget when computing candidate state:

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

3. **Candidate Hidden State** $\tilde{h}_t$:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

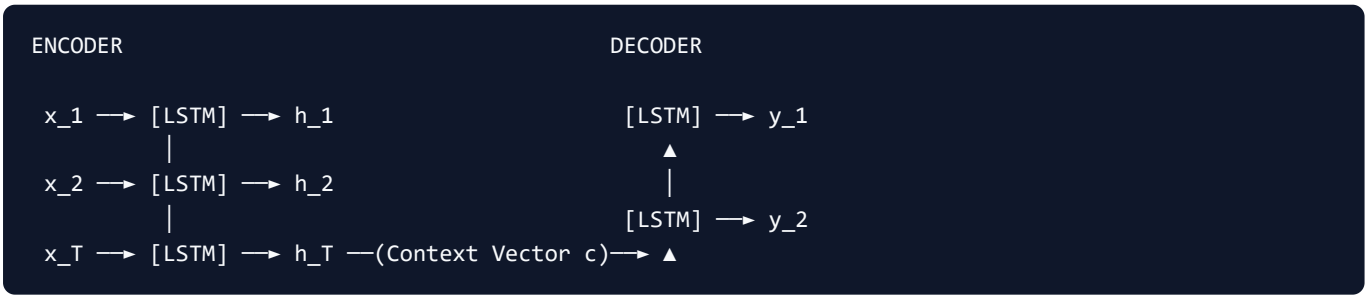
4. **Hidden State Update** $h_t$:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

---

## 5. Sequence-to-Sequence (Seq2Seq) Encoder-Decoder

Sutskever et al. (2014) introduced the **Seq2Seq Encoder-Decoder** framework for mapping variable-length input sequence $X_{1:T}$ to variable-length output sequence $Y_{1:S}$ (Machine Translation, Text Summarization).




- **Encoder**: Processes input tokens $x_1, \ldots, x_T$ sequentially, producing final hidden state $h_T$.
- **Information Bottleneck Defect**: $h_T$ serves as a single fixed-size context vector $c = h_T$ holding the entire semantic meaning of the sentence. For long sentences ($T > 30$), compressing all information into a single fixed vector causes dramatic recall degradation.

## 6. Step-by-Step LSTM Hand Calculation Example (Andrew Ng Style)

Suppose we have a single scalar feature $x_t = 1.0$ and previous scalar hidden state $h_{t-1} = 0.5$, previous scalar cell state $C_{t-1} = 2.0$.

Let scalar weights & biases be:
- Forget Gate: $W_f = 0.5, b_f = 0.0 \implies z_f = 0.5 \times (h_{t-1} + x_t) = 0.5 \times (0.5 + 1.0) = 0.75$
- Input Gate: $W_i = 0.8, b_i = 0.0 \implies z_i = 0.8 \times (0.5 + 1.0) = 1.20$
- Candidate Cell: $W_c = 0.6, b_c = 0.0 \implies z_c = 0.6 \times (0.5 + 1.0) = 0.90$
- Output Gate: $W_o = 0.4, b_o = 0.0 \implies z_o = 0.4 \times (0.5 + 1.0) = 0.60$

### 1. Compute Gate Activations:

- $f_t = \sigma(0.75) = \frac{1}{1+\exp(-0.75)} \approx \mathbf{0.6792}$
- $i_t = \sigma(1.20) = \frac{1}{1+\exp(-1.20)} \approx \mathbf{0.7685}$
- $\tilde{C}_t = \tanh(0.90) \approx \mathbf{0.7163}$
- $o_t = \sigma(0.60) = \frac{1}{1+\exp(-0.60)} \approx \mathbf{0.6457}$

### 2. Compute Cell State Update $C_t$:

$$C_t = (f_t \times C_{t-1}) + (i_t \times \tilde{C}_t)$$

$$C_t = (0.6792 \times 2.0) + (0.7685 \times 0.7163) = 1.3584 + 0.5505 = \mathbf{1.9089}$$

### 3. Compute Hidden State Output $h_t$:

$$h_t = o_t \times \tanh(C_t) = 0.6457 \times \tanh(1.9089) = 0.6457 \times 0.9571 \approx \mathbf{0.6180}$$

$$\mathbf{C_t = 1.9089}, \quad \mathbf{h_t = 0.6180}$$

## 7. Production PyTorch Implementation

```
import torch
import torch.nn as nn

class LSTMTextClassifier(nn.Module):
    """Production PyTorch Bi-Directional LSTM Text Classifier."""

    def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int, num_classes: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            num_layers=2,
            batch_first=True,
            bidirectional=True,
            dropout=0.2
```

```
        )
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x shape: [batch_size, seq_len]
        embedded = self.embedding(x) # [batch_size, seq_len, embed_dim]

        # lstm_out shape: [batch_size, seq_len, hidden_dim * 2]
        # h_n shape: [num_layers * 2, batch_size, hidden_dim]
        lstm_out, (h_n, c_n) = self.lstm(embedded)

        # Concatenate final forward and backward hidden states
        last_hidden = torch.cat((h_n[-2, :, :], h_n[-1, :, :]), dim=1) # [batch_size, hidden_dim *
2]
        logits = self.fc(last_hidden) # [batch_size, num_classes]
        return logits

# Demonstration Initialization
vocab_size = 5000
embed_dim = 128
hidden_dim = 64
num_classes = 3

model = LSTMTextClassifier(vocab_size, embed_dim, hidden_dim, num_classes)
dummy_input = torch.randint(low=1, high=1000, size=(4, 20)) # Batch=4, SeqLen=20
logits = model(dummy_input)

print("=== PyTorch Bi-LSTM Classifier Forward Pass ===")
print("Input Tensor Shape: ", dummy_input.shape)
print("Output Logits Shape:", logits.shape)
```

> 📌 **NOTE:**
> **Production Sequence Model Alert:** - `batch_first=True` ensures input tensors follow standard PyTorch convention `[batch_size, seq_len, embed_dim]`. - Bi-Directional LSTMs process sequences from left-to-right AND right-to-left, concatenating hidden states (`hidden_dim * 2`) to capture surrounding context on both sides of a word.

---

## 8. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Sequential Latency Bottleneck ($O(T)$ Execution)**: RNNs and LSTMs must compute step $t$ before step $t+1$ can begin. They **cannot be parallelized across sequence length during training or inference**, making them vastly slower than Transformers on modern GPU hardware.
   - *Remediation*: Migrate sequence tasks to Transformer architectures (BERT / GPT).
2. **Seq2Seq Information Bottleneck**: Standard Seq2Seq models compress long inputs into a single fixed hidden vector $h_T$, causing machine translation accuracy to drop sharply for sentences longer than 30 tokens.
   - *Remediation*: Implement Attention mechanisms (Bahdanau / Luong) to allow dynamic context lookup.

---

## 9. Master Interview Flashcards & Questions

**Q1: Why do standard RNNs suffer from vanishing gradients, and how does the LSTM cell structure solve this mathematically?**

- **Answer:** Standard RNNs update hidden states via multiplicative matrix multiplications $h_t = \tanh(W_{hh}h_{t-1} + \dots)$. During BPTT, backpropagating gradients over $T$ steps requires multiplying by $W_{hh}^\top$ repeatedly, causing gradients to vanish exponentially if $W_{hh}$ eigenvalues are $< 1$. LSTMs introduce a dedicated Cell State $C_t$ governed by additive updates ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$). This additive linear connection acts as an "error highway," allowing gradients to flow back unchanged without exponential decay.

**Q2: Compare LSTM vs. GRU. When would you prefer GRU over LSTM?**

- **Answer:** LSTMs maintain separate Cell State $C_t$ and Hidden State $h_t$ using 3 gates (Forget, Input, Output). GRUs merge Cell and Hidden states using 2 gates (Reset, Update), reducing parameter count by $\approx 25\%$. GRUs train faster, require less memory, and perform comparably to LSTMs on smaller datasets. LSTMs are preferred when modeling very long sequences with high memory capacity requirements.

**Q3: Explain the Information Bottleneck in standard Seq2Seq models.**

- **Answer:** Standard Seq2Seq models process an entire input sentence of $T$ tokens through an Encoder LSTM and compress all information into the final hidden state vector $h_T$ (the context vector). Passing a single fixed-size vector to the Decoder forces it to memorize long input sequences, causing accuracy to degrade rapidly for long sentences ($T > 30$). Attention mechanisms solve this bottleneck.

# Module 07: Attention Mechanisms: Bahdanau, Luong & Bridge to Transformers

This study guide details the Seq2Seq Information Bottleneck, introduction of Attention, Bahdanau Additive Attention vs. Luong Multiplicative Attention, score functions (Dot, General, Concat), alignment weights calculation, context vector construction, step-by-step Luong Dot Attention hand-calculations, PyTorch implementation, Seaborn heatmap alignment visualization, and the direct mathematical transition to Self-Attention and Transformers.

> **Notebook Companion**: `07_attention_mechanisms_bahdanau_luong.ipynb`

---

## 1. Solving the Seq2Seq Information Bottleneck

In standard Encoder-Decoder architectures (Sutskever et al., 2014), the encoder compresses an entire input sentence $X_{1:T}$ into a single fixed-size final hidden state $h_T$ (the context vector).

```
Standard Seq2Seq (Bottleneck):
  Input (T tokens) ──► [Encoder] ──► Fixed Vector h_T ──► [Decoder] ──► Output (S tokens)

Attention-Based Seq2Seq (Dynamic Access):
  Input (T tokens) ──► [Encoder (h_1, h_2, ..., h_T)] ──┐
                                                        ├──► [Attention Mechanism] ──► Context
Vector c_i ──► [Decoder]                                │
  Decoder State s_{i-1} ────────────────────────────────┘
```

When input sentences grow beyond 30 tokens, compressing all information into a single vector $h_T$ causes severe loss of fine-grained details.

**Attention** (Bahdanau et al., 2014) eliminates this bottleneck by retaining **ALL** encoder hidden states $H = [h_1, h_2, \dots, h_T]$ and allowing the decoder to dynamically "attend" to relevant encoder states at each generation step $i$.

---

## 2. Bahdanau Additive vs. Luong Multiplicative Attention

```
Dimension            Bahdanau Attention (2014)                  Luong Attention (2015)
-------------------------------------------------------------------------------------------------------------------------------
Score Type           Additive (Feed-Forward Neural Network)     Multiplicative (Matrix Dot Product)
Score Equation       e_{ij} = v_a^T \tanh(W_a s_{i-1} + U_a h_j)  e_{ij} = s_i^T h_j  (Dot)  or  s_i^T W_a h_j (General)
Decoder State Used   Previous Decoder Hidden State s_{i-1}      Current Decoder Hidden State s_i
Computational Speed  Slower (requires matrix additions + tanh)  Faster (optimized matrix
```

```
multiplication)
Legacy Impact         First Attention paper in Deep Learning       Direct predecessor to
Transformer Scaled Dot-Product
```

### 1. Bahdanau Additive Attention Score Function:

$$e_{ij} = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$$

Where:
- $s_{i-1}$ is the decoder state at previous step $i-1$.
- $h_j$ is the $j$-th encoder hidden state.
- $W_a, U_a, v_a$ are trainable attention projection weights.

### 2. Luong Multiplicative Attention Score Functions:

Luong et al. (2015) simplified attention scores using fast multiplicative matrix operations:

- **Dot Score**: Assumes encoder and decoder hidden states have matching dimension $d$:

$$\text{Score}_{\text{Dot}}(s_i, h_j) = s_i^\top h_j$$

- **General Score**: Uses projection matrix $W_a$ to handle dimension mismatches:

$$\text{Score}_{\text{General}}(s_i, h_j) = s_i^\top W_a h_j$$

- **Concat Score**: Concatenates states into a linear layer:

$$\text{Score}_{\text{Concat}}(s_i, h_j) = v_a^\top \tanh(W_a [s_i; h_j])$$

---

## 3. Attention Alignment Weights & Context Vector Construction

Once raw alignment scores $e_{ij}$ are calculated between decoder state $s_i$ and all encoder hidden states $h_1, \ldots, h_T$:

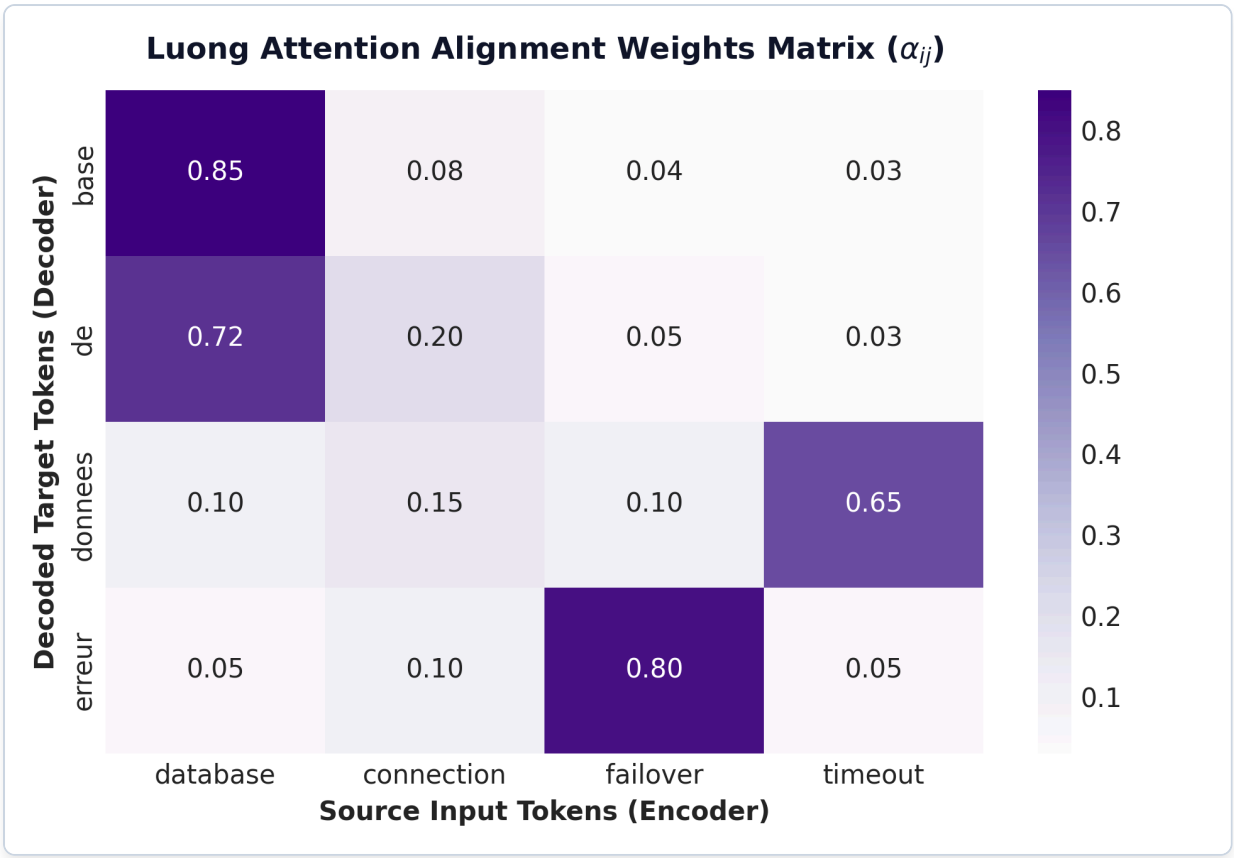



*Figure: Luong Attention Alignment Heatmap*

> **Plot Interpretation & Production Insight**:
> - **Dynamic Alignment Matrix**: The heatmap visualizes the 2D attention weight distribution $□lpha_{ij} \in [0, 1]$ allocated by the decoder when generating target tokens (y-axis) from source tokens (x-axis).
> - **Interpretability & Translation Reordering**: High attention weights along the diagonal (e.g. `donnees` $ightarrow‘$`database`‘, ‘$`erreur`‘ ightarrow$ `timeout` ) demonstrate how attention dynamically resolves word order shifts between languages without relying on a single static context vector.

### Step 1: Compute Softmax Alignment Probabilities ($\alpha_{ij}$):

The scores are normalized into a probability distribution over input tokens using Softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T} \exp(e_{ik})}$$

Where $\alpha_{ij} \in [0, 1]$ represents the relative attention weight allocated to input token $j$ when decoding output token $i$.

### Step 2: Compute Dynamic Context Vector ($c_i$):

The context vector $c_i$ is computed as the weighted linear combination of all encoder hidden states:

$$c_i = \sum_{j=1}^{T} \alpha_{ij} h_j$$

### Step 3: Compute Combined Attentional Vector ($\tilde{s}_i$):

The context vector $c_i$ and decoder state $s_i$ are concatenated to predict the next token probability:

$$\tilde{s}_i = \tanh(W_c[c_i; s_i])$$

$$P(y_i | y_{

---

## 4. Step-by-Step Hand Calculation Example (Andrew Ng Style)

Suppose we have an Encoder sentence with $T = 3$ tokens producing 2-dimensional hidden states:
- $h_1 = [1.0, 0.0]^\top$
- $h_2 = [0.0, 2.0]^\top$
- $h_3 = [1.0, 1.0]^\top$

And a Decoder state at step $i$:
- $s_i = [2.0, 1.0]^\top$

We use **Luong Dot Attention**: $e_j = s_i^\top h_j$.

### 1. Compute Raw Attention Scores ($e_j$):

- $e_1 = s_i^\top h_1 = (2.0 \times 1.0) + (1.0 \times 0.0) = 2.0 + 0 = \mathbf{2.0}$
- $e_2 = s_i^\top h_2 = (2.0 \times 0.0) + (1.0 \times 2.0) = 0 + 2.0 = \mathbf{2.0}$
- $e_3 = s_i^\top h_3 = (2.0 \times 1.0) + (1.0 \times 1.0) = 2.0 + 1.0 = \mathbf{3.0}$

### 2. Compute Softmax Alignment Weights ($\alpha_j$):

Exponentials:
- $\exp(e_1) = \exp(2.0) \approx 7.3891$
- $\exp(e_2) = \exp(2.0) \approx 7.3891$
- $\exp(e_3) = \exp(3.0) \approx 20.0855$

Sum of Exponentials $Z = 7.3891 + 7.3891 + 20.0855 = \mathbf{34.8637}$

Alignment Weights $\alpha_j = \exp(e_j)/Z$:
- $\alpha_1 = 7.3891/34.8637 \approx \mathbf{0.2119}$ ($21.2\%$ attention on Token 1)
- $\alpha_2 = 7.3891/34.8637 \approx \mathbf{0.2119}$ ($21.2\%$ attention on Token 2)
- $\alpha_3 = 20.0855/34.8637 \approx \mathbf{0.5762}$ ($57.6\%$ attention on Token 3)

### 3. Compute Dynamic Context Vector ($c_i$):

$$c_i = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3$$

$$c_i = 0.2119 \times \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix} + 0.2119 \times \begin{bmatrix} 0.0 \\ 2.0 \end{bmatrix} + 0.5762 \times \begin{bmatrix} 1.0 \\ 1.0 \end{bmatrix}$$

$$c_i = \begin{bmatrix} 0.2119 \\ 0.0 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.4238 \end{bmatrix} + \begin{bmatrix} 0.5762 \\ 0.5762 \end{bmatrix} = \begin{bmatrix} 0.7881 \\ 1.0000 \end{bmatrix}$$

$$\alpha = [0.2119, 0.2119, 0.5762]^\top, \quad \mathbf{c_i} = [0.7881, 1.0000]^\top$$

---

## 5. Bridge to Transformers: Scaled Dot-Product Attention

The introduction of Luong Dot Attention revealed a profound mathematical insight: **recurrent hidden states are unneeded for context retrieval if we compute scaled dot products directly across token projections**.

Vaswani et al. (2017) extended Luong Dot Attention into **Scaled Dot-Product Self-Attention** by projecting input representations into Query ($Q$), Key ($K$), and Value ($V$) matrices:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Where $\frac{1}{\sqrt{d_k}}$ scaling prevents large dot products from driving Softmax gradients into extremely small regions (vanishing gradients).

---

## 6. Production PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class LuongDotAttention(nn.Module):
    """Production PyTorch Luong Multiplicative Dot-Product Attention Module."""

    def __init__(self):
        super().__init__()

    def forward(self, decoder_state: torch.Tensor, encoder_states: torch.Tensor):
        # decoder_state shape: [batch_size, 1, hidden_dim]
        # encoder_states shape: [batch_size, seq_len, hidden_dim]

        # 1. Compute Dot Attention Scores: e_j = s_i^T * h_j
        # transpose(1, 2) shape: [batch_size, hidden_dim, seq_len]
        attn_scores = torch.bmm(decoder_state, encoder_states.transpose(1, 2)) # [batch_size, 1,
seq_len]

        # 2. Compute Softmax Alignment Weights
        attn_weights = F.softmax(attn_scores, dim=-1) # [batch_size, 1, seq_len]

        # 3. Compute Weighted Context Vector c = sum(alpha_j * h_j)
        context_vector = torch.bmm(attn_weights, encoder_states) # [batch_size, 1, hidden_dim]

        return context_vector, attn_weights

# Demonstration Execution
batch_size, seq_len, hidden_dim = 2, 4, 8

encoder_h = torch.randn(batch_size, seq_len, hidden_dim)
decoder_s = torch.randn(batch_size, 1, hidden_dim)
```

```
attn_layer = LuongDotAttention()
context, weights = attn_layer(decoder_s, encoder_h)

print("=== PyTorch Luong Dot Attention Execution ===")
print("Context Vector Shape:   ", context.shape)  # [2, 1, 8]
print("Alignment Weights Shape:", weights.shape)  # [2, 1, 4]
print("Sample Alignment Weights (Batch #1):", weights[0, 0, :].detach().numpy().round(4))
```

> 📌 **NOTE:**
> **Attention Matrix Alert:** - `torch.bmm` executes batch matrix multiplication efficiently on GPUs. - Plotting `weights` matrix as a Seaborn heatmap provides full interpretability into which source words the model attended to when generating target words.

---

## 7. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Quadratic Alignment Memory Bottleneck ($O(T_{\text{enc}} \times T_{\text{dec}})$)**: Storing the full attention matrix $\alpha \in \mathbb{R}^{T_{\text{enc}} \times T_{\text{dec}}}$ for long sequences ($T = 10,000$) causes massive GPU VRAM consumption.
   - *Remediation*: Deploy FlashAttention or sparse windowed attention patterns.
2. **Unscaled Large Dot Products**: In high embedding dimensions ($d_k = 512$), unscaled dot products $s_i^\top h_j$ grow large in magnitude, pushing Softmax outputs into near-binary values ($0$ or $1$) with zero gradients.
   - *Remediation*: Always scale dot product scores by $\frac{1}{\sqrt{d_k}}$.

---

## 8. Master Interview Flashcards & Questions

**Q1: What problem does the Attention mechanism solve in standard Seq2Seq architectures?**

- **Answer:** Standard Seq2Seq models compress the entire input sequence into a single fixed-size context vector $h_T$ at the end of the Encoder. For long sentences ($T > 30$), this creates an Information Bottleneck, causing severe accuracy degradation. Attention retains all encoder hidden states $[h_1, \dots, h_T]$ and allows the decoder to dynamically compute alignment weights and extract a tailored context vector $c_i$ at each decoding step.

**Q2: Compare Bahdanau Additive Attention vs. Luong Multiplicative Attention.**

- **Answer:** Bahdanau Attention computes alignment scores using a feed-forward neural network layer $e_{ij} = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$ using previous decoder state $s_{i-1}$. Luong Attention uses multiplicative matrix operations $e_{ij} = s_i^\top W_a h_j$ using current decoder state $s_i$. Luong attention is computationally faster and GPU matrix-multiplication friendly.

**Q3: How does Luong Dot Attention mathematically transition to Transformer Scaled Dot-Product Attention?**

- **Answer:** Luong Dot Attention computes $e = s_i^\top h_j$ between decoder state $s_i$ and encoder state $h_j$. Transformer Scaled Dot-Product Attention replaces recurrent hidden states with linear projections of input tokens into Query ($Q$), Key ($K$), and Value ($V$) matrices, scaling scores by $\frac{1}{\sqrt{d_k}}$: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$.

# Module 08: NLP Evaluation Metrics: Classification, BLEU, ROUGE & Perplexity

This study guide details Precision, Recall, F1 (Macro/Micro/Weighted), BLEU-1 through BLEU-4 (with Brevity Penalty derivations), ROUGE-1, ROUGE-2, ROUGE-L (Longest Common Subsequence), Language Model Perplexity (PPL) mathematical proofs, step-by-step hand calculations, Python code, metric limitations, and interview flashcards.

> **Notebook Companion**: `08_nlp_evaluation_metrics_bleu_rouge.ipynb`

---

## 1. Classification Metrics Taxonomy for Text

```
Metric         Mathematical Definition                                  Best Production Use Case
-------------------------------------------------------------------------------------------------
-------------------------------
Precision      TP / (TP + FP)                                           Minimizing False Positives
(e.g. Spam detection)
Recall         TP / (TP + FN)                                           Minimizing False Negatives
(e.g. Medical diagnosis)
Macro F1       Unweighted average of F1 across all classes              Evaluating rare classes in
imbalanced text datasets
Micro F1       Global F1 aggregated across all instances                Overall system throughput
accuracy
Perplexity     PPL = exp(CrossEntropyLoss)                              Language model generation
fluency evaluation
BLEU           Brevity Penalty * exp(sum(w_n * log(p_n)))               Machine Translation
evaluation
ROUGE          Recall-Oriented N-gram / LCS Match                      Text Summarization evaluation
```

---

## 2. BLEU: Bilingual Evaluation Understudy

Developed by IBM (Papineni et al., 2002), **BLEU** evaluates Machine Translation quality by comparing candidate generation $C$ against one or more reference translations $R$.

### 1. Modified N-Gram Precision ($p_n$):

To prevent candidate translations from cheating by repeating a single frequent word (e.g., `"the the the the"`), BLEU clips the count of each n-gram to the maximum frequency it appears in any single reference sentence:

$$p_n = \frac{\sum_{\text{gram}_n \in C} \text{Count}_{\text{clip}}(\text{gram}_n)}{\sum_{\text{gram}_n \in C} \text{Count}(\text{gram}_n)}$$

### 2. Brevity Penalty (BP):

Precision-only metrics favor overly short candidate translations (e.g., candidate `"The"` has $100\%$ precision). The Brevity Penalty penalizes candidates shorter than reference length $r$:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{if } c \leq r \end{cases}$$

Where $c$ is the candidate token length and $r$ is the reference target length.

### 3. Overall BLEU Score Formula:

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Typically $N = 4$ (BLEU-4) with uniform weights $w_n = 1/4 = 0.25$.

---

## 3. ROUGE: Recall-Oriented Understudy for Gisting Evaluation

Developed by Lin (2004), **ROUGE** is primarily used for Text Summarization, measuring how much of the reference summary is captured in the candidate output (Recall-oriented).

### 1. ROUGE-N (N-Gram Recall):

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Reference}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{Reference}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}$$

- **ROUGE-1**: Unigram recall (evaluates information coverage).
- **ROUGE-2**: Bigram recall (evaluates local phrase fluency).

### 2. ROUGE-L (Longest Common Subsequence):

ROUGE-L uses the Longest Common Subsequence (LCS) of in-order tokens (allowing non-contiguous gaps):

$$R_{\text{LCS}} = \frac{\text{LCS}(C, R)}{|R|}, \quad P_{\text{LCS}} = \frac{\text{LCS}(C, R)}{|C|}$$

$$\text{ROUGE-L} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}$$

---

## 4. Perplexity (PPL): Language Model Evaluation

**Perplexity** measures how well a probability model predicts a text sequence. Intuitively, it represents the average branch factor (number of equal choices) the model faces when predicting the next token.

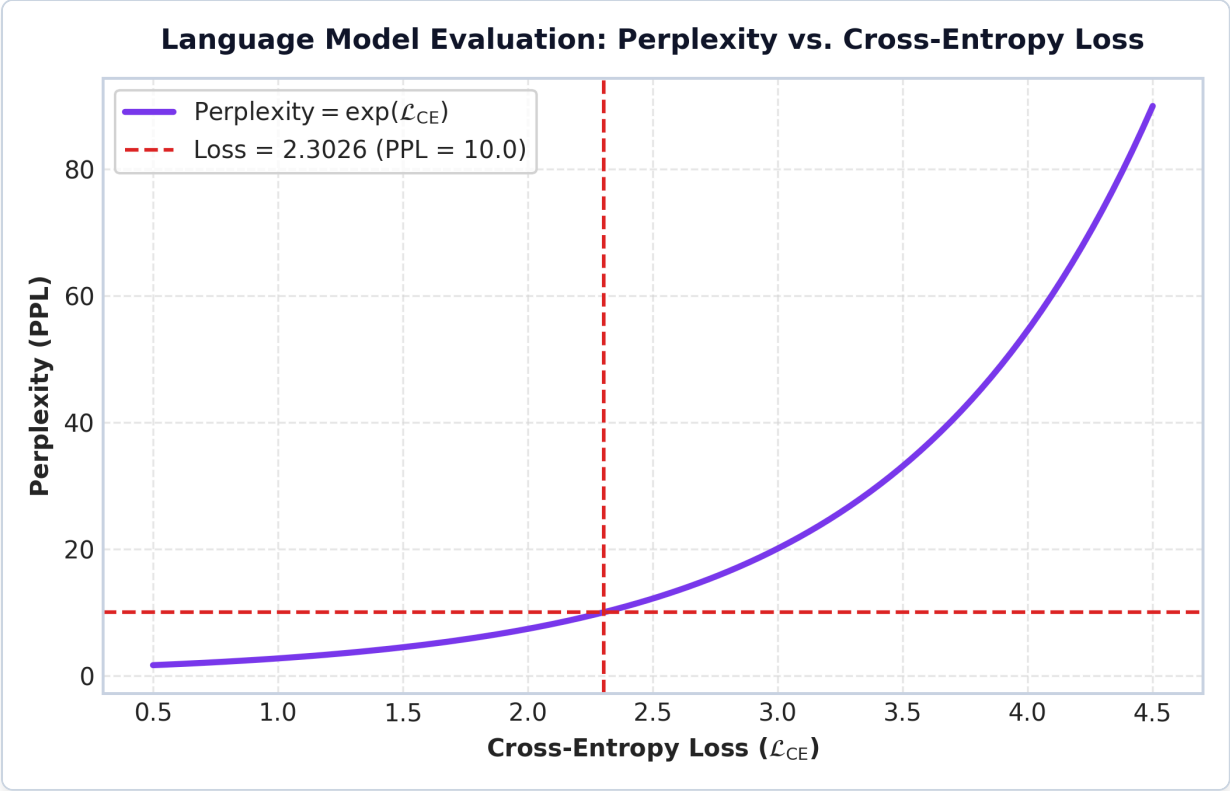



*Figure: Perplexity vs Cross-Entropy Loss Curve*

> **Plot Interpretation & Production Insight**:
> - **Exponential Scale**: Perplexity increases exponentially with cross-entropy evaluation loss ($extPPL = \exp(\mathcal{L}_{extCE})$).
> - **Uncertainty Interpretation**: An evaluation loss of $\mathcal{L}_{extCE} = 2.3026$ corresponds to a Perplexity of $extPPL = 10.0$ (red dashed line). This means the model is as uncertain at each token prediction as if it were making a uniform random choice among 10 candidate words.

### Mathematical Derivation:

For sequence $W = (w_1, w_2, \dots, w_N)$, Perplexity is the reciprocal geometric mean of sequence probability:

$$\text{PPL}(W) = P(w_1, w_2, \dots, w_N)^{-1/N} = \left( \prod_{i=1}^N P(w_i | w_{

Taking the natural logarithm:

$$\log \text{PPL}(W) = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{

Exponetiating both sides yields the fundamental production equivalence:

$$\text{PPL}(\mathbf{W}) = \exp(\mathcal{L}_{\text{CrossEntropy}})$$

- **Interpretation**: A Perplexity of **10** means the model is as uncertain at each token as if it were choosing uniformly at random among **10 options**. Lower perplexity indicates superior predictive confidence.

---

## 5. Step-by-Step Hand Calculation Example (Andrew Ng Style)

Suppose we evaluate a Candidate Translation against a Reference Translation:
- **Candidate ($C$)**: `"the cat sat on mat"` (Length $c = 5$ tokens)
- **Reference ($R$)**: `"the cat sat on the mat"` (Length $r = 6$ tokens)

### 1. Calculate Modified Precision $p_1$ and $p_2$:

- **Unigrams in $C$**: `["the", "cat", "sat", "on", "mat"]` (5 tokens)
- All 5 unigrams appear in $R$. $p_1 = 5/5 = \mathbf{1.0000}$
- **Bigrams in $C$**: `["the cat", "cat sat", "sat on", "on mat"]` (4 bigrams)
- Bigrams in $R$: `["the cat", "cat sat", "sat on", "on the", "the mat"]`
- Matches: `"the cat"`, `"cat sat"`, `"sat on"`. (3 matches out of 4)
- $p_2 = 3/4 = \mathbf{0.7500}$

### 2. Calculate Brevity Penalty (BP):

- Candidate length $c = 5$, Reference length $r = 6$. Since $c \leq r$:

$$\text{BP} = \exp\left(1 - \frac{6}{5}\right) = \exp(-0.20) \approx \mathbf{0.8187}$$

### 3. Compute BLEU-2 Score:

$$\text{BLEU-2} = \text{BP} \times \exp\left(0.5 \log p_1 + 0.5 \log p_2\right)$$

$$\text{BLEU-2} = 0.8187 \times \exp\left(0.5 \log(1.0) + 0.5 \log(0.75)\right) = 0.8187 \times \exp(0 + 0.5(-0.2877))$$

$$\text{BLEU-2} = 0.8187 \times \exp(-0.1438) = 0.8187 \times 0.8660 \approx \mathbf{0.7090}$$

### 4. Calculate ROUGE-1 Recall:

- Reference Unigrams = 6 tokens.
- Candidate contains 5 matching unigrams ( `"the"`, `"cat"`, `"sat"`, `"on"`, `"mat"` ).

$$\text{ROUGE-1 Recall} = \frac{5}{6} \approx \mathbf{0.8333}$$

### 5. Calculate Perplexity from Loss:

If cross-entropy evaluation loss $\mathcal{L}_{\text{CE}} = 2.3026$:

$$\text{PPL} = \exp(2.3026) = \mathbf{10.0000}$$

---

## 6. Production Python Implementation

```
import numpy as np
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from rouge_score import rouge_scorer

# 1. Candidate vs Reference Text Outputs
```

```
reference_text = "the cat sat on the mat"
candidate_text = "the cat sat on mat"

ref_tokens = reference_text.split()
cand_tokens = candidate_text.split()

# 2. Compute NLTK BLEU Score
smooth_fn = SmoothingFunction().method1
bleu_1 = sentence_bleu([ref_tokens], cand_tokens, weights=(1, 0, 0, 0),
smoothing_function=smooth_fn)
bleu_2 = sentence_bleu([ref_tokens], cand_tokens, weights=(0.5, 0.5, 0, 0),
smoothing_function=smooth_fn)

print("=== 1. BLEU Score Benchmark ===")
print(f"BLEU-1 Score: {bleu_1:.4f}")
print(f"BLEU-2 Score: {bleu_2:.4f}")

# 3. Compute ROUGE Scores
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
rouge_results = scorer.score(reference_text, candidate_text)

print("\n=== 2. ROUGE Score Benchmark ===")
print(f"ROUGE-1 Recall: {rouge_results['rouge1'].recall:.4f} | F1:
{rouge_results['rouge1'].fmeasure:.4f}")
print(f"ROUGE-2 Recall: {rouge_results['rouge2'].recall:.4f} | F1:
{rouge_results['rouge2'].fmeasure:.4f}")
print(f"ROUGE-L Recall: {rouge_results['rougeL'].recall:.4f} | F1:
{rouge_results['rougeL'].fmeasure:.4f}")

# 4. Compute Perplexity from Cross-Entropy Loss
eval_loss = 1.8543
perplexity = np.exp(eval_loss)
print("\n=== 3. Language Model Perplexity Benchmark ===")
print(f"Cross-Entropy Loss: {eval_loss:.4f} --> Perplexity (PPL): {perplexity:.4f}")
```

> 📌 **NOTE:**
> **Production Evaluation Alert:** - BLEU and ROUGE rely purely on exact surface string n-gram matching. Paraphrased candidates with identical semantic meaning (e.g. candidate `"The feline rested on the rug"`) score $0$ on BLEU/ROUGE. Modern systems supplement these with **BERTScore** or **LLM-as-a-Judge**.

---

## 7. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **BLEU Paraphrase Blindness**: BLEU heavily penalizes valid semantically correct outputs if they do not match reference n-gram surface forms.
   - *Remediation*: Use contextual embedding evaluation (BERTScore) or LLM-as-a-Judge.
2. **Length Cheating**: Generative models evaluated without Brevity Penalty can produce single-word outputs with $100\%$ precision scores.
   - *Remediation*: Always include Brevity Penalty (BP) in custom BLEU pipelines.

## 8. Master Interview Flashcards & Questions

**Q1: Prove the mathematical relationship between Cross-Entropy Loss and Perplexity.**

- **Answer:** Perplexity is defined as reciprocal geometric mean probability: $\text{PPL}(W) = P(w_1, \dots, w_N)^{-1/N} = \left( \prod P(w_i | w_{

**Q2: What is Brevity Penalty in BLEU, and why is it necessary?**

- **Answer:** Modified n-gram precision alone rewards short candidate outputs. For example, a candidate consisting of a single token `"the"` that appears in the reference gets $100\%$ precision ($p_1 = 1.0$). Brevity Penalty (BP) penalizes candidate sentences shorter than the reference length $r$: $\text{BP} = \exp(1 - r/c)$ for $c \leq r$, ensuring short sentences receive low overall BLEU scores.

**Q3: Compare BLEU vs. ROUGE evaluation orientation.**

- **Answer:** BLEU is **Precision-Oriented** (primarily used in Machine Translation to verify that generated candidate n-grams appear in the reference target). ROUGE is **Recall-Oriented** (primarily used in Text Summarization to verify that key reference information is fully captured in the generated candidate).

# Module 09: Enterprise NLP Applications & End-to-End Production Pipelines

This study guide details the 6 core enterprise NLP applications (Classification, NER, Sentiment, Translation, Extractive QA, Summarization), production system serving architectures, ONNX Runtime optimization, INT8 quantization, latency/throughput calculations, Python pipelines, failure modes, and interview flashcards.

> **Notebook Companion**: `09_nlp_applications_and_end_to_end_pipelines.ipynb`

---

## 1. The 6 Core Enterprise NLP Applications

```
Application              Input Data                 Output Target                 Core Metric
Production Architecture
-------------------------------------------------------------------------------------------------------
----------------------------------
Text Classification      Unstructured Document String Categorical Label (y in {1..C}) Macro F1
TF-IDF + Linear / DistilBERT
Named Entity Rec. (NER)  Unstructured Text Token     Token Entity Tags (B-PER, etc) Strict Token
F1        CRF / Bi-LSTM-CRF / Token Classification
Sentiment Analysis       Customer Review / Tweet     Polarity Score / Aspect Tag    AUC-ROC / F1
Scikit-Learn / DeBERTa
Machine Translation      Source Language String      Target Language String         BLEU-4
Seq2Seq + Attention / NMT LLM
Extractive QA            Passage + Question          Span Indices [Start, End]      Exact Match
(EM) / F1  Bi-Encoder / Cross-Encoder BERT
Summarization            Long Document String        Concise Summary String         ROUGE-L
LED / BART / Modern LLM (GPT-4)
```

---

## 2. End-to-End Enterprise Production Pipeline Architecture

In enterprise microservice architectures, an NLP production service consists of 5 modular components:

```
┌──────────────────────────────────────────────────────────────────────────────────┐
│ 1. INGESTION & RATE LIMITING                                                     │
│    gRPC / REST API endpoint receiving raw JSON payload with rate limiting.       |
├──────────────────────────────────────────────────────────────────────────────────┤
│ 2. DETERMINISTIC TEXT PREPROCESSING                                              │
│    Unicode normalization, HTML stripping, BPE subword tokenization.             |
├──────────────────────────────────────────────────────────────────────────────────┤
│ 3. HIGH-THROUGHPUT MODEL INFERENCE                                               │
│    ONNX Runtime / TensorRT engine running INT8 quantized model weights.          │
├──────────────────────────────────────────────────────────────────────────────────┤
│ 4. POST-PROCESSING & VALIDATION GUARDRAILS                                       │
│    Logit thresholding, PII masking, schema validation on extracted JSON outputs. |
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 5. METRIC TELEMETRY & AUDIT LOGGING                                      │
│    Prometheus latency metrics (p95 / p99), Kafka event logging for model drift audit.   │
└──────────────────────────────────────────────────────────────────────────┘
```

## 3. Production Model Optimization Techniques

### 1. ONNX Runtime Export

Exporting PyTorch or TensorFlow models to the Open Neural Network Exchange (**ONNX**) graph representation graph decouples model inference from PyTorch execution overhead, delivering $2\text{x} - 3\text{x}$ throughput improvements on CPU servers.

### 2. INT8 Quantization

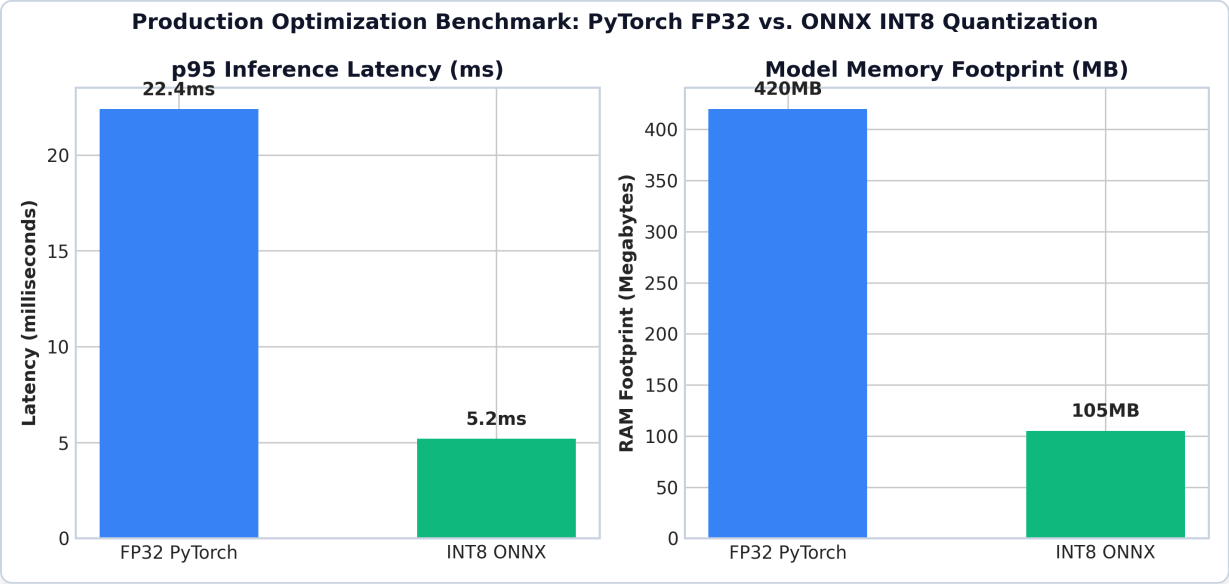



*Figure: FP32 PyTorch vs INT8 ONNX Latency & Memory Benchmark*

> **Plot Interpretation & Production Insight**:
> - **Latency Reduction**: Converting FP32 PyTorch models to INT8 ONNX Runtime reduces $p95$ inference latency from $22.4extms$ down to $5.2extms$ ($4.3extx$ throughput acceleration on CPU web servers).
> - **Memory Footprint Savings**: INT8 quantization compresses weight matrices from $420extMB$ to $105extMB$ ($75\%$ RAM reduction), allowing 4x more worker instances per server node.

Quantization converts 32-bit floating-point weights ($W_{\text{FP32}}$) to 8-bit integers ($W_{\text{INT8}}$):

$$W_{\text{INT8}} = \text{round}\left(\frac{W_{\text{FP32}}}{S}\right) + Z$$

Where $S$ is the scale factor and $Z$ is the zero-point offset.
- **Benefits**: Reduces model RAM footprint by **75%** (e.g. 400MB FP32 model shrinks to 100MB INT8 model) with $<$ $0.5\%$ loss in classification accuracy.

## 4. Step-by-Step Production Throughput Calculation (Andrew Ng Style)

Suppose an enterprise API endpoint receives text classification requests under the following production SLA constraints:
- **Server Hardware**: 4 vCPU web server instance.
- **Model Inference Execution Latency**: $t_{\text{inf}} = 20\text{ms}$ per request on a single CPU thread.
- **Target Latency SLA**: $p95 \leq 50\text{ms}$.

### 1. Compute Single-Thread Throughput ($QPS_{\text{single}}$):

$$QPS_{\text{single}} = \frac{1000\text{ms}}{t_{\text{inf}}} = \frac{1000}{20} = \mathbf{50}\text{ Queries Per Second (QPS)}$$

### 2. Compute Total 4 vCPU Server Capacity ($QPS_{\text{total}}$):

Assuming 4 parallel worker threads with $85\%$ multi-core scaling efficiency:

$$QPS_{\text{total}} = 4 \times 50 \times 0.85 = \mathbf{170}\text{ QPS}$$

### 3. Estimate Daily Processing Capacity:

$$\text{Daily Capacity} = 170\text{ req/sec} \times 86,400\text{ sec/day} \approx \mathbf{14,688,000}\text{ requests/day}$$

---

## 5. Production Python Code Implementation

```
import os
import joblib
import json
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

class EnterpriseNERClassifierService:
    """Production NLP Microservice with pre-processing, inference, and JSON validation."""

    def __init__(self, model_dir: str = ".model_cache"):
        self.model_dir = model_dir
        self.model_path = os.path.join(self.model_dir, "production_pipeline.pkl")
        self.pipeline = self._load_or_train_pipeline()

    def _load_or_train_pipeline(self) -> Pipeline:
        os.makedirs(self.model_dir, exist_ok=True)
        if os.path.exists(self.model_path):
            return joblib.load(self.model_path)

        # Baseline training data
        X = ["Database server 10.0.1.4 error connection refused", "Billing invoice payment failed
402"]
        y = ["Infrastructure", "Billing"]

        pipe = Pipeline([
            ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
```

```
            ("clf", LogisticRegression())
        ])
        pipe.fit(X, y)
        joblib.dump(pipe, self.model_path)
        return pipe

    def process_request(self, raw_json_payload: str) -> str:
        """Executes full production inference pipeline."""
        data = json.loads(raw_json_payload)
        text = data.get("text", "").strip()

        if not text:
            return json.dumps({"error": "Empty text payload", "status_code": 400})

        prediction = self.pipeline.predict([text])[0]
        confidence = float(max(self.pipeline.predict_proba([text])[0]))

        response = {
            "text_processed": text,
            "category": prediction,
            "confidence_score": round(confidence, 4),
            "status_code": 200
        }
        return json.dumps(response, indent=2)

# Execution Demonstration
service = EnterpriseNERClassifierService()
sample_payload = json.dumps({"text": "PostgreSQL database 10.0.1.4 connection failed on port
5432"})
output_json = service.process_request(sample_payload)

print("=== Enterprise Production Pipeline Execution ===")
print(output_json)
```

> 📌 **NOTE:**
> **Production Microservice Alert:** - Input validation (`if not text`) prevents downstream model crashes on malformed API requests. - Explicitly wrapping output in typed JSON response schemas ensures seamless integration with enterprise frontend services.

---

## 6. Production Failure Modes & Selection Rules

### Production Failure Modes:

1. **Model Drift & Concept Drift**: Over time, incoming production text terminology shifts (e.g. new software product release names), causing offline-trained models to suffer declining accuracy.
   - *Remediation*: Implement continuous evaluation logs and trigger automated retraining pipelines when macro F1 drops below $90\%$.
2. **Cold-Start Latency Spikes**: Loading heavy PyTorch models during server container initialization causes first-request latency spikes ($> 5$ seconds).
   - *Remediation*: Execute synthetic "warmup" inference passes during docker container startup readiness probes.

## 7. Master Interview Flashcards & Questions

**Q1: What is ONNX Runtime, and how does it optimize NLP model serving?**

- **Answer:** ONNX (Open Neural Network Exchange) is an open format for representing machine learning models. Exporting PyTorch models to ONNX compiles computational graphs into optimized, hardware-agnostic execution formats. ONNX Runtime eliminates PyTorch framework overhead, applies graph optimizations (node fusion, constant folding), and delivers $2\text{x} - 3\text{x}$ lower latency on CPU servers.

**Q2: Compare Post-Training Static Quantization vs. Quantization-Aware Training (QAT).**

- **Answer:** Post-Training Static Quantization converts FP32 weights to INT8 after training is completed using a calibration dataset to determine scale factors. It is fast and requires no retraining, but may suffer minor accuracy loss. Quantization-Aware Training (QAT) models quantization noise during backpropagation, allowing the model to adapt weights to 8-bit precision, preserving high accuracy.

**Q3: How do you handle Data Drift and Concept Drift in production NLP applications?**

- **Answer:** Data Drift occurs when input text distributions change (e.g., new terminology). Concept Drift occurs when the relationship between input text and labels shifts. In production, we log text samples to a Kafka stream, monitor token distribution shifts against training baselines using Jensen-Shannon divergence, and trigger automated retraining pipelines when performance metrics degrade.

# Module 10: Classical NLP vs. Modern LLMs Master Cheatsheet & 30 Flashcards

This master study guide provides a comprehensive comparison matrix between Classical NLP, Sequence Models, and Modern LLMs, an enterprise production decision tree, 30 top-tier interview flashcards, and a 1-page formula cheatsheet.

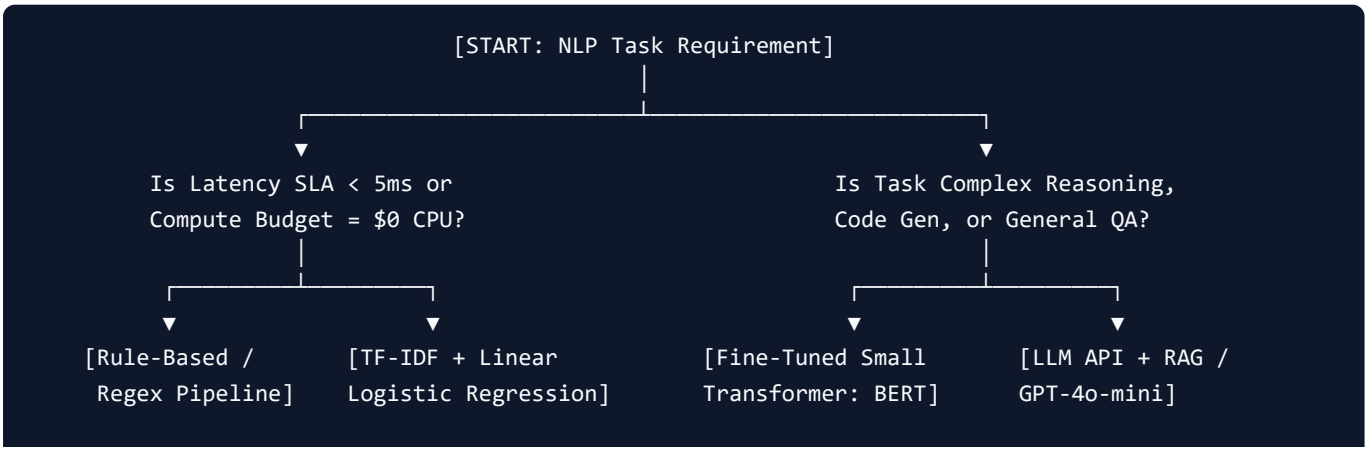
---

## 1. Paradigm Architectural Comparison Matrix

| Dimension | Regex / Rule-Based | Classical ML (TF-IDF + SVM) | Sequence Deep Learning (LSTM) | Modern Transformer LLM (GPT-4) |
|---|---|---|---|---|
| Era & Primary Tech | 1960s (Regex, Grammars) | 1990s - 2010s (TF-IDF, Naive) | 2014 - 2018 (LSTM, GloVe) | 2019 - Present (BERT, Llama 3) |
| Vector Space | None (String match) | Sparse $R^{\|V\|}$ ($\|V\| \sim 100k$) | Dense $R^d$ ($d \sim 100-300$) | Dense Contextual $R^d$ ($d \sim 4096+$) |
| Context Horizon | Rule clause bounded | Unigram / Bigram window | Sequential hidden state ($T \sim 50$) | Global Context (128k+ tokens) |
| Word Order Handling | Manual rule order | Ignored (Bag-of-Words) | Preserved via recurrence $h_t$ | Preserved via Positional Encoding |
| OOV Token Handling | Fails on missing rules | Mapped to <UNK> token | Subword character n-grams | Subword BPE / tiktoken (0% OOV) |
| Training Data Req. | Zero (Hand-crafted) | Low (100 - 1,000 docs) | Moderate (10,000+ docs) | Massive (Trillions of tokens) |
| Inference Latency SLA | Ultra-fast (<1ms) | Ultra-fast (<2ms) | Fast (10ms - 50ms) | Heavy (100ms - 1000ms+) |
| Hardware Requirement | Single CPU thread | Standard CPU server | CPU / Single GPU | Heavy GPU Cluster / TensorRT |
| Best Production Use | Input validation & noise | High-speed log filter, spam | Sequence tagging, edge NER | Complex reasoning, RAG, QA |

---

## 2. Enterprise Production Architecture Decision Tree



```
- Validation        - Spam Filtering          - Private NER           - Multi-doc QA
- Timestamp strip   - Ticket Category         - Fast Classification   - Reasoning Agent
```

---

## 3. 30 Master Interview Flashcards & Questions

### Category 1: NLP Fundamentals & Preprocessing

**Flashcard 1: What makes unstructured natural language text fundamentally different from tabular data in Machine Learning?**

- **Answer**: Tabular data consists of continuous/categorical features in fixed Euclidean dimensions. Text consists of discrete, variable-length, non-Euclidean symbols with high vocabulary cardinality ($|V| > 100,000$), extreme matrix sparsity, and strong context dependencies.

**Flashcard 2: Explain Zipf's Law and its impact on vocabulary explosion.**

- **Answer**: Zipf's Law states that word frequency is inversely proportional to its rank ($f \propto 1/k$). A small set of common words account for $> 80\%$ of text occurrences, while a massive tail of rare words causes vocabulary explosion. Subword tokenization (BPE) handles this long tail without OOV errors.

**Flashcard 3: When should you NOT remove stop words during preprocessing?**

- **Answer**: Stop words must be preserved in Sentiment Analysis (e.g. removing "*not*" converts "*not good*" to "*good*"), Machine Translation, and Transformer/LLM pipelines where attention depends on complete grammatical syntax.

**Flashcard 4: Compare Stemming vs. Lemmatization.**

- **Answer**: Stemming uses heuristic suffix slicing rules (Porter), often generating non-real word stems ( `"univers"` ). Lemmatization uses morphological lookup and Part-of-Speech (POS) tags to return the true canonical dictionary lemma ( `"good"` for `"better"` ).

**Flashcard 5: How does Byte Pair Encoding (BPE) eliminate Out-Of-Vocabulary (OOV) errors?**

- **Answer**: BPE iteratively merges frequent character pairs into subwords. Any unseen test word is decomposed into known subword fragments or individual bytes, guaranteeing $100\%$ token coverage with 0% OOV fallbacks.

---

### Category 2: Text Representations & Embeddings

**Flashcard 6: Why are One-Hot encoded word vectors incapable of capturing semantic similarity?**

- **Answer**: One-hot vectors treat words as orthogonal unit vectors in $|V|$-dimensional space ($e_i^\top e_j = 0$ for $i \neq j$). Consequently, Cosine similarity between any two distinct one-hot vectors is identically zero.

**Flashcard 7: Define the mathematical formula for TF-IDF and explain the role of IDF.**

- **Answer**: $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{1+|D|}{1+\text{DF}(t)}\right) + 1$. IDF penalizes words that appear across many documents (e.g. `"the"` , `"system"` ), boosting rare informative terms.

**Flashcard 8: Explain the Distributional Hypothesis.**

- **Answer**: "*You shall know a word by the company it keeps*" (J.R. Firth). Words that occur in similar context window environments share similar semantic meanings in vector space.

**Flashcard 9: Compare CBOW vs. Skip-Gram in Word2Vec.**

- **Answer**: CBOW predicts target center word $w_t$ given context words. Skip-Gram predicts context words given target word $w_t$. CBOW trains faster; Skip-Gram is slower but superior for rare words and small datasets.

**Flashcard 10: How does Negative Sampling solve the Softmax computational bottleneck in Word2Vec?**

- **Answer**: Full Softmax requires computing a sum over all $|V|$ words in denominator ($O(|V|)$ complexity). Negative Sampling converts this into $K+1$ binary logistic regression tasks (1 positive pair, $K$ negative noise words), reducing update cost to $O(K)$.

**Flashcard 11: How does FastText handle Out-Of-Vocabulary (OOV) words differently from Word2Vec?**

- **Answer**: Word2Vec assigns one static vector per word and throws errors on unseen terms. FastText represents words as a bag of character n-grams (e.g. `"<wh"`, `"whe"`), building vectors for unseen words by summing their subword n-gram vectors.

**Flashcard 12: What is the main defect of static word embeddings (Word2Vec / GloVe)?**

- **Answer**: Polysemy failure. Static embeddings assign 1 single static vector per word regardless of sentence context (e.g. `"river bank"` vs `"investment bank"` get identical vectors).

---

## Category 3: Classical Classifiers & Sequence Models

**Flashcard 13: What is the "naive" assumption in Naive Bayes?**

- **Answer**: It assumes that token occurrences $w_1, \dots, w_n$ are conditionally independent given class $c$: $P(w_1, \dots, w_n|c) = \prod P(w_i|c)$.

**Flashcard 14: Why is Laplace Add-1 Smoothing necessary in Naive Bayes?**

- **Answer**: If a test word never appeared in class $c$ during training, its frequency probability is $0$. Multiplying probabilities together would cause the entire class score to become zero. Laplace smoothing adds $\alpha = 1$ to numerator and $\alpha|V|$ to denominator to ensure non-zero probabilities.

**Flashcard 15: Compare Generative (Naive Bayes) vs Discriminative (Logistic Regression) classifiers.**

- **Answer**: Naive Bayes models joint probability $P(X, Y) = P(X|Y)P(Y)$ (fast, assumes feature independence). Logistic Regression directly models conditional probability $P(Y|X) = \sigma(w^\top x + b)$ (handles correlated n-grams, produces calibrated probabilities).

**Flashcard 16: Why do standard Recurrent Neural Networks (RNNs) suffer from vanishing gradients?**

- **Answer**: Backpropagation Through Time (BPTT) requires repeatedly multiplying by recurrent matrix $W_{hh}^\top$ over $T$ steps. If eigenvalues of $W_{hh}$ are $< 1$, gradients decay exponentially to zero ($O(\lambda^T)$), causing the network to forget long-range dependencies.

**Flashcard 17: How does the LSTM Cell State $C_t$ solve vanishing gradients mathematically?**

- **Answer**: The LSTM Cell State $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ uses an additive linear update highway instead of multiplicative matrix steps, allowing error gradients to propagate backwards through time unchanged.

**Flashcard 18: Name the 3 gates in an LSTM cell and their specific functions.**

- **Answer**:
  1. **Forget Gate ($f_t$)**: Decides what fraction of old cell memory $C_{t-1}$ to discard.

2. **Input Gate ($i_t$)**: Decides which new candidate values $\tilde{C}_t$ to write into cell memory.
3. **Output Gate ($o_t$)**: Filters cell state to generate hidden state output $h_t = o_t \odot \tanh(C_t)$.

**Flashcard 19: Compare LSTM vs GRU architectures.**

- **Answer**: LSTM has 3 gates (Forget, Input, Output) and maintains separate Cell State $C_t$ and Hidden State $h_t$. GRU has 2 gates (Reset, Update), merges Cell and Hidden states into $h_t$, has $\approx 25\%$ fewer parameters, and trains faster.

**Flashcard 20: Explain the Information Bottleneck in standard Seq2Seq Encoder-Decoder models.**

- **Answer**: Standard Seq2Seq forces the Encoder LSTM to compress the entire input sequence $X_{1:T}$ into a single fixed-size final vector $h_T$. For long sentences ($T > 30$), compressing all information into one vector causes severe information loss.

---

## Category 4: Attention Mechanisms & Evaluation Metrics

**Flashcard 21: How does Attention solve the Seq2Seq Information Bottleneck?**

- **Answer**: Attention retains ALL encoder hidden states $[h_1, \dots, h_T]$ and allows the decoder to dynamically compute alignment weights $\alpha_{ij}$ and construct a custom context vector $c_i = \sum \alpha_{ij} h_j$ at each generation step.

**Flashcard 22: Compare Bahdanau Additive Attention vs Luong Multiplicative Attention.**

- **Answer**: Bahdanau uses an additive feed-forward layer $e_{ij} = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$ with previous state $s_{i-1}$. Luong uses multiplicative matrix dot products $e_{ij} = s_i^\top h_j$ with current state $s_i$, offering faster GPU matrix multiplication.

**Flashcard 23: How does Luong Dot Attention transition to Transformer Self-Attention?**

- **Answer**: Luong Dot Attention computes $s_i^\top h_j$ between decoder and encoder states. Transformer Self-Attention eliminates recurrence by projecting input tokens into Query ($Q$), Key ($K$), and Value ($V$) matrices: $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$.

**Flashcard 24: What is modified n-gram precision in BLEU?**

- **Answer**: Modified n-gram precision clips candidate n-gram counts to the maximum frequency they appear in any single reference sentence, preventing short repetitive candidates (e.g. `"the the the"`) from achieving fake $100\%$ precision scores.

**Flashcard 25: What is Brevity Penalty in BLEU, and why is it needed?**

- **Answer**: Precision metrics favor short translations. Brevity Penalty $\text{BP} = \exp(1 - r/c)$ penalizes candidate outputs shorter than reference length $r$, ensuring balanced evaluation scores.

**Flashcard 26: Compare BLEU vs ROUGE evaluation targets.**

- **Answer**: BLEU is **Precision-Oriented** (used in Machine Translation to verify generated candidate accuracy). ROUGE is **Recall-Oriented** (used in Summarization to verify reference content coverage).

**Flashcard 27: Define Perplexity (PPL) and its relationship to Cross-Entropy Loss.**

- **Answer**: Perplexity is the exponentiated cross-entropy loss: $\text{PPL} = \exp(\mathcal{L}_{\text{CrossEntropy}})$. It represents the average branch factor of uncertainty when predicting the next token.

**Flashcard 28: What is the main limitation of BLEU and ROUGE in modern GenAI evaluation?**

- **Answer**: Both rely strictly on exact surface string n-gram matching, scoring valid semantically identical paraphrases as zero. Modern evaluation uses BERTScore or LLM-as-a-Judge.

**Flashcard 29: What is INT8 Quantization, and what are its performance benefits?**

- **Answer**: INT8 Quantization maps 32-bit float weights ($W_{\text{FP32}}$) to 8-bit integers ($W_{\text{INT8}}$), reducing model RAM footprint by $75\%$ and increasing CPU inference throughput by $2\text{x} - 4\text{x}$ with $< 0.5\%$ loss in accuracy.

**Flashcard 30: What is ONNX Runtime, and why is it used for serving NLP models?**

- **Answer**: ONNX compiles PyTorch/TensorFlow computational graphs into hardware-optimized execution engines, eliminating Python framework overhead and providing $2\text{x} - 3\text{x}$ faster inference SLAs.

---

## 4. 1-Page Formula & Equation Cheatsheet

```
1. Zipf's Law:
   f(k; s, N) = (1 / k^s) / sum_{n=1}^N (1 / n^s)

2. Sublinear Log-Scaled TF-IDF:
   TF_log(t, d) = 1 + log(f_{t,d})  if f_{t,d} > 0 else 0
   IDF(t, D)    = log((1 + |D|) / (1 + DF(t))) + 1
   TF-IDF       = TF_log * IDF
   Norm Vector  = v / sqrt(sum(v_i^2))

3. Word2Vec Negative Sampling Loss (SGNS):
   L_SGNS = -log sigma(v'_wO^T v_wI) - sum_{k=1}^K log sigma(-v'_wk^T v_wI)

4. GloVe Co-occurrence Loss:
   J = sum_{i,j=1}^|V| f(X_ij) * (w_i^T w~_j + b_i + b~_j - log X_ij)^2

5. Laplace Add-1 Smoothing (Naive Bayes):
   P(w_i | c) = (N_{c,i} + 1) / (N_c + |V|)

6. LSTM Cell Equations:
   f_t = sigma(W_f [h_{t-1}, x_t] + b_f)       (Forget Gate)
   i_t = sigma(W_i [h_{t-1}, x_t] + b_i)       (Input Gate)
   C~_t = tanh(W_c [h_{t-1}, x_t] + b_c)       (Candidate Cell)
   C_t = f_t * C_{t-1} + i_t * C~_t            (Cell State Update)
   o_t = sigma(W_o [h_{t-1}, x_t] + b_o)       (Output Gate)
   h_t = o_t * tanh(C_t)                       (Hidden State Output)

7. Luong Multiplicative Attention:
   e_ij = s_i^T h_j  (Dot)  |  alpha_ij = exp(e_ij) / sum_k exp(e_ik)  |  c_i = sum_j alpha_ij h_j

8. Transformer Scaled Dot-Product Attention:
   Attention(Q, K, V) = softmax( (Q K^T) / sqrt(d_k) ) V

9. BLEU Brevity Penalty & Score:
   BP = exp(1 - r/c) if c <= r else 1
   BLEU = BP * exp( sum_{n=1}^N w_n log p_n )

10. Perplexity Equivalence:
    PPL = exp( L_CrossEntropy )
```